

**ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI**  
**TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG**



**BÁO CÁO BTL THUỘC HỌC PHẦN IT6094**  
**TRÍ TUỆ NHÂN TẠO**

**XÂY DỰNG HỆ THỐNG CHATBOT HỎI ĐÁP TÀI LIỆU SỬ DỤNG KIẾN  
TRÚC RAG**

GVHD : ThS. Trần Văn Huân

Lớp học phần : 20252IT6097002

Hà Nội - 2026

**ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI**  
**TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG**



**BÁO CÁO BTL THUỘC HỌC PHẦN IT6094:**  
**TRÍ TUỆ NHÂN TẠO**

**XÂY DỰNG HỆ THỐNG CHATBOT HỎI**  
**ĐÁP TÀI LIỆU SỬ DỤNG KIẾN**  
**TRÚC RAG**

GVHD : Ths.Trần Văn Huân

Nhóm – Lớp : 03 - 2025IT6094

Thành viên : Lê Thanh Trúc  
Lương Tuấn Anh  
Hoàng Gia Bảo  
Nguyễn Tiến Hùng

Hà Nội - 2026

## LỜI CẢM ƠN

Sau khi hoàn thành bài tiểu luận “Xây dựng hệ thống chatbot hỏi đáp tài liệu sử dụng kiến trúc RAG”, chúng em xin gửi lời cảm ơn chân thành đến tất cả những cá nhân và tổ chức đã hỗ trợ trong suốt quá trình thực hiện.

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Ths.Trần Văn Huân - người đã tận tình hướng dẫn, cung cấp kiến thức chuyên môn và đưa ra những góp ý quý báu giúp chúng em hoàn thiện bài tiểu luận một cách tốt nhất.

Chúng em cũng xin cảm ơn Trường Công nghệ thông tin và Truyền thông đã tạo điều kiện thuận lợi cho nhóm chúng em về môi trường học tập và tài liệu tham khảo. Đồng thời, chúng em xin gửi lời cảm ơn đến các bạn trong lớp học phần Trí tuệ nhân tạo đã cùng trao đổi, thảo luận và hỗ trợ nhóm trong suốt quá trình thực hiện đề tài.

Và cuối cùng, chúng em xin kính chúc thầy sức khỏe và thành công trong sự nghiệp. Chúc các bạn luôn dồi dào sức khỏe, đạt được nhiều thành công trong quá trình học tập.

**Chúng em xin chân thành cảm ơn!**

## MỤC LỤC

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| LỜI CẢM ƠN .....                                                            | 1  |
| DANH MỤC CÁC THUẬT NGỮ, KÝ HIỆU VÀ CÁC CHỮ VIẾT TẮT .....                   | 4  |
| DANH MỤC HÌNH ẢNH.....                                                      | 7  |
| DANH MỤC BẢNG BIỂU .....                                                    | 8  |
| MỞ ĐẦU .....                                                                | 9  |
| 1. Lý do chọn đề tài .....                                                  | 9  |
| 2. Mục tiêu của đề tài.....                                                 | 10 |
| 3. Đối tượng và phạm vi nghiên cứu .....                                    | 10 |
| 4. Cấu trúc của báo cáo .....                                               | 11 |
| CHƯƠNG 1: LÝ THUYẾT CƠ SỞ .....                                             | 1  |
| 1.1 Kiến thức cơ bản về xử lý ngôn ngữ tự nhiên và trí tuệ nhân tạo .....   | 1  |
| 1.1.1 Khái niệm về mô hình ngữ lớn .....                                    | 1  |
| 1.1.2 Tổng quan về mô hình Google Gemini API.....                           | 1  |
| 1.1.3 Kỹ thuật nhúng văn bản và SentenceTransformers .....                  | 2  |
| 1.1.4 Khái niệm cơ bản về cơ sở dữ liệu Vector .....                        | 3  |
| 1.2 Tổng quan về kiến trúc RAG .....                                        | 3  |
| 1.2.1 RAG là gì và tại sao cần sử dụng RAG? .....                           | 3  |
| 1.2.2 Các khối chức năng cốt lõi trong kiến trúc RAG .....                  | 4  |
| 1.2.3 Quy trình xử lý dữ liệu đầu vào .....                                 | 4  |
| 1.2.4 Quy trình truy xuất dữ liệu và tạo câu trả lời .....                  | 5  |
| 1.2.5 Vai trò của FAISS trong hệ thống RAG .....                            | 6  |
| 1.2.6 Vai trò của Framework LangChain trong hệ thống RAG.....               | 7  |
| 1.3 Tổng quan về Flask Framework.....                                       | 7  |
| 1.3.1 Giới thiệu Flask.....                                                 | 7  |
| 1.3.2 Mô hình Client – Server trong ứng dụng Web .....                      | 7  |
| 1.4 Kết luận chương 1 .....                                                 | 7  |
| CHƯƠNG 2: THIẾT KẾ CHI TIẾT VÀ CÁC THÀNH PHẦN HỆ THỐNG<br>CHATBOT RAG ..... | 9  |
| 2.1 Kiến trúc dữ liệu và thành phần tìm kiếm vector FAISS .....             | 9  |
| 2.1.1 Mô hình phân cấp cấu trúc tài liệu đầu vào (PDF, DOCX, PPTX).....     | 9  |
| 2.1.2 Cơ chế trích xuất nội dung văn bản đa định dạng .....                 | 10 |
| 2.1.3 Phân lớp lưu trữ và tìm kiếm tương đồng với thư viện FAISS.....       | 11 |

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| 2.2 Kiến trúc và chức năng của Chatbot AI Assistant .....                    | 12 |
| 2.2.1 Sơ đồ kiến trúc tổng thể của hệ thống Chatbot.....                     | 12 |
| 2.2.2 Cơ chế truy xuất dữ liệu liên quan .....                               | 13 |
| 2.2.3 Tích hợp Google Gemini API để sinh câu trả lời .....                   | 14 |
| 2.2.4 Chức năng kiểm soát thông tin và hạn chế hiện tượng “ảo tưởng” ...     | 15 |
| 2.3 Thiết kế giao diện Web Flask.....                                        | 15 |
| 2.3.1 Thiết kế giao diện người dùng .....                                    | 15 |
| 2.3.2 Luồng tương tác người dùng và hệ thống .....                           | 16 |
| 2.4 Kết luận chương 2.....                                                   | 16 |
| CHƯƠNG 3: TRIỂN KHAI ỨNG DỤNG VÀ THỰC NGHIỆM KẾT QUẢ ..                      | 17 |
| 3.1 Môi trường phát triển và công nghệ sử dụng.....                          | 17 |
| 3.2 Cấu trúc mã nguồn và giải pháp công nghệ (Python Flask).....             | 18 |
| 3.2.1 Thiết kế kiến trúc Backend ứng dụng với Python Flask.....              | 21 |
| 3.2.2 Xây dựng giao diện Chatbot hiện đại trực quan với HTML .....           | 21 |
| 3.3 Xây dựng các chức năng chính .....                                       | 24 |
| 3.3.1 Xử lý và lập chỉ mục tài liệu.....                                     | 24 |
| 3.3.2 Tìm kiếm ngữ nghĩa bằng FAISS .....                                    | 24 |
| 3.3.3 Sinh câu trả lời bằng Gemini API.....                                  | 24 |
| 3.3.4 Xây dựng giao diện Web với Flask.....                                  | 25 |
| 3.4 Các vấn đề kỹ thuật và đánh giá chất lượng hệ thống.....                 | 25 |
| 3.4.1 Kịch bản kiểm thử ứng dụng với các file tài liệu lớn .....             | 25 |
| 3.4.2 Đánh giá độ chính xác của câu trả lời dựa trên nội dung tài liệu ..... | 26 |
| 3.4.3 Tối ưu hóa tốc độ phản hồi và xử lý lỗi khi gọi Gemini API .....       | 26 |
| 3.5 Kết quả đạt được.....                                                    | 26 |
| 3.6 Kết luận chương 3.....                                                   | 27 |
| KẾT LUẬN.....                                                                | 28 |
| 1. Kết luận .....                                                            | 28 |
| 2. Định hướng phát triển.....                                                | 28 |
| TÀI LIỆU THAM KHẢO .....                                                     | 30 |

**DANH MỤC CÁC THUẬT NGỮ, KÝ HIỆU VÀ CÁC CHỮ VIẾT TẮT**

| STT | Chữ viết tắt | Tên đầy đủ                        | Nội dung                                            |
|-----|--------------|-----------------------------------|-----------------------------------------------------|
| 1   | AI           | Artificial Intelligence           | Trí tuệ nhân tạo                                    |
| 2   | NLP          | Natural Language Processing       | Xử lý ngôn ngữ tự nhiên                             |
| 3   | LLM          | Large Language Model              | Mô hình ngôn ngữ lớn                                |
| 4   | RAG          | Retrieval-Augmented Generation    | Kỹ thuật kết hợp truy xuất dữ liệu và sinh nội dung |
| 5   | API          | Application Programming Interface | Giao diện lập trình ứng dụng                        |
| 6   | UI           | User Interface                    | Giao diện người dùng                                |
| 7   | UX           | User Experience                   | Trải nghiệm người dùng                              |
| 8   | PDF          | Portable Document Format          | Định dạng tài liệu PDF                              |
| 9   | DOCX         | Document Open XML                 | Định dạng tài liệu Microsoft Word                   |
| 10  | PPTX         | PowerPoint Open XML Presentation  | Định dạng tệp trình chiếu PowerPoint                |
| 11  | FAISS        | Facebook AI Similarity Search     | Thư viện tìm kiếm tương đồng vector                 |

|    |           |                             |                                           |
|----|-----------|-----------------------------|-------------------------------------------|
| 12 | Vector DB | Vector Ko                   | Cơ sở dữ liệu lưu trữ vector              |
| 13 | Embedding | Text Embedding              | Biểu diễn văn bản dưới dạng vector số     |
| 14 | ST        | Sentence Transformer        | Mô hình tạo vector ngữ nghĩa cho văn bản  |
| 15 | Gemini    | Google Gemini               | Mô hình ngôn ngữ lớn của Google           |
| 16 | JSON      | JavaScript Object Notation  | Định dạng trao đổi dữ liệu                |
| 17 | HTTP      | HyperText Transfer Protocol | Giao thức truyền tải siêu văn bản         |
| 18 | GPU       | Graphics Processing Unit    | Bộ xử lý đồ họa                           |
| 19 | CPU       | Central Processing Unit     | Bộ xử lý trung tâm                        |
| 20 | RAM       | Random Access Memory        | Bộ nhớ truy cập ngẫu nhiên                |
| 21 | Chatbot   | Chat Robot                  | Hệ thống đối thoại tự động với người dùng |
| 22 | Prompt    | Prompt Engineering          | Nội dung hướng dẫn gửi đến mô hình AI     |
| 23 | Context   | Context Window              | Ngữ cảnh được cung cấp cho mô hình        |

|    |                   |                       |                                              |
|----|-------------------|-----------------------|----------------------------------------------|
| 24 | Top-k             | Top-k Retrieval       | Kỹ thuật lấy k kết quả phù hợp nhất          |
| 25 | Cosine Similarity | Cosine Similarity     | Độ đo mức độ tương đồng giữa các vector      |
| 26 | Streamlit         | Streamlit Framework   | Framework Python xây dựng giao diện Web AI   |
| 27 | Pickle (.pkl)     | Python Pickle         | Định dạng lưu trữ đối tượng Python           |
| 28 | Index             | Vector Index          | Cấu trúc chỉ mục phục vụ tìm kiếm vector     |
| 29 | Chunk             | Text Chunk            | Đoạn văn bản được chia nhỏ từ tài liệu       |
| 30 | Retrieval         | Information Retrieval | Quá trình truy xuất thông tin từ kho dữ liệu |



## DANH MỤC HÌNH ẢNH

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Hình 1.1 Sơ đồ hoạt động Google Gemini API.....                             | 2  |
| Hình 1.2 Quá trình chuyển đổi văn bản thành Vecto .....                     | 2  |
| Hình 1.3 So sánh kiến trúc Rag và hệ thống truyền thống .....               | 3  |
| Hình 1.4 Sơ đồ tổng thể hệ thống RAG .....                                  | 4  |
| Hình 1.5 Sơ đồ quy trình tiền xử lý và chia nhỏ văn bản trong hệ thống..... | 5  |
| Hình 1.6 Quy trình hoạt động của Chatbot Rag .....                          | 6  |
| Hình 2.1 Mô hình phân cấp dữ liệu .....                                     | 9  |
| Hình 2.2 Các thư viện Python dùng để trích xuất văn bản .....               | 10 |
| Hình 2.3 Cơ chế lưu trữ và tìm kiếm Faiss .....                             | 11 |
| Hình 2.4 Sơ đồ kiến trúc tổng thể hệ thống.....                             | 13 |
| Hình 3.1 Cấu trúc mã nguồn.....                                             | 18 |
| Hình 3.2 Quy trình hoạt động mã nguồn hệ thống Chatbot Rag.....             | 21 |
| Hình 3.3 Giao diện chính của hệ thống Chatbot .....                         | 22 |
| Hình 3.4 Chức năng tải tài liệu lên hệ thống.....                           | 22 |
| Hình 3.5 Giao diện khu vực hội thoại.....                                   | 23 |
| Hình 3.6 Kết quả trả lời câu hỏi dựa trên nội dung tài liệu .....           | 23 |
| Hình 3.7 Kết quả câu trả lời bằng Google API .....                          | 25 |

**DANH MỤC BẢNG BIỂU**

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Bảng 3.1 Trình bày tóm tắt các công nghệ được sử dụng trong đề tài..... | 17 |
| Bảng 3.2 Chức năng các tệp mã nguồn trong hệ thống .....                | 19 |
| Bảng 3.3 Kết quả đạt được .....                                         | 26 |

## MỞ ĐẦU

### 1. Lý do chọn đề tài

Trong kỷ nguyên số hóa và sự bùng nổ thông tin hiện nay, việc quản trị, khai thác và tra cứu tri thức nội bộ đóng vai trò sống còn đối với sự phát triển của mọi tổ chức, doanh nghiệp và cơ sở đào tạo. Theo một nghiên cứu nổi tiếng của McKinsey, nhân viên văn phòng trung bình phải dành tới 1,8 giờ mỗi ngày (tương đương gần 20% thời gian làm việc tuần) chỉ để tìm kiếm và thu thập thông tin cần thiết nhằm hoàn thành công việc. Việc tra cứu thủ công trên các kho tài liệu truyền thống khổng lồ (như PDF, Word, tài liệu nội bộ) không chỉ gây chậm trễ tiến độ, tăng chi phí vận hành mà còn dễ dẫn đến sai sót do bỏ lỡ các thông tin cập nhật. Nguyên nhân chủ yếu đến từ việc thiếu hụt một công cụ tương tác thông minh, có khả năng hiểu và phản hồi ngôn ngữ tự nhiên một cách tức thời.

Sự ra đời của các Mô hình ngôn ngữ lớn (LLM) như GPT, Gemini hay Claude đã mở ra một cuộc cách mạng trong việc xử lý ngôn ngữ tự nhiên. Tuy nhiên, khi áp dụng trực tiếp LLM vào việc hỏi đáp dữ liệu nội bộ, các tổ chức phải đối mặt với hai thách thức cốt lõi: hiện tượng "ảo tưởng" (hallucination) — khi mô hình tự bịa ra thông tin không có thực — và giới hạn về dữ liệu đóng (LLM không được huấn luyện trên các tài liệu mật hoặc tài liệu đặc thù của tổ chức). Để giải quyết triệt để hạn chế này, kiến trúc RAG (Retrieval-Augmented Generation - Tạo sinh tăng cường truy xuất) nổi lên như một giải pháp đột phá. Bằng cách kết hợp giữa khả năng tìm kiếm ngữ nghĩa siêu tốc trên cơ sở dữ liệu Vector và năng lực tổng hợp ngôn ngữ của LLM, RAG giúp chatbot chỉ câu trả lời bám sát vào tài liệu gốc được cung cấp, tối ưu hóa độ chính xác và bảo mật thông tin.

Đối với một hệ thống hỏi đáp tài liệu, mục tiêu cốt lõi ngay từ đầu là giúp người dùng (nhân viên, sinh viên, khách hàng) tối ưu hóa thời gian tra cứu các văn bản quy chế, tài liệu kỹ thuật hay hướng dẫn sử dụng. Các luồng nghiệp vụ từ việc nạp dữ liệu, cắt nhỏ văn bản (chunking), nhúng vector (embedding) cho đến truy xuất ngữ cảnh và cấu trúc prompt đều có tính logic và kỹ thuật rất cao. Việc nghiên cứu và làm chủ kiến trúc RAG không chỉ giúp giải quyết bài toán tối ưu hóa hiệu suất khai thác tri thức trong thực tiễn, mà còn là cơ hội để tiếp cận với những công nghệ tiên tiến nhất trong lĩnh vực Trí tuệ nhân tạo.

Xuất phát từ những lý do trên, nhóm chúng em lựa chọn đề tài “Xây dựng hệ thống Chatbot hỏi đáp tài liệu sử dụng kiến trúc RAG” nhằm hiện thực hóa giải pháp công nghệ thông minh, đảm bảo tính khả thi cao và mang lại giá trị ứng dụng thiết thực trong việc chuyển đổi số công tác quản trị tri thức.

## 2. Mục tiêu của đề tài

- Nghiên và ứng dụng AI: Làm chủ các công nghệ cốt lõi bao gồm Mô hình ngôn ngữ lớn, kiến trúc RAG, framework LangChain, và cơ sở dữ liệu vector FAISS để xử lý tài liệu.
- Phát triển Chatbot Thông minh: Xây dựng thành công hệ thống hỏi đáp có khả năng đọc, hiểu và truy xuất thông tin chính xác từ văn bản gốc.
- Hạn chế thông tin bịa đặt: Đảm bảo câu trả lời được tổng hợp chính xác, hoàn toàn dựa trên nội dung tài liệu do người dùng cung cấp.
- Tối ưu hóa trải nghiệm: Thiết kế giao diện Web trực quan, hỗ trợ tải lên nhiều định dạng tài liệu (PDF, DOCX, PPTX) và tương tác bằng ngôn ngữ tự nhiên.
- Lập trình và tích hợp: Vận dụng ngôn ngữ Python kết hợp với các thư viện/công cụ Flask, Streamlit, SentenceTransformers và tích hợp Google Gemini API để hoàn thiện hệ thống.

## 3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu:

- Nghiên cứu ứng dụng Trí tuệ nhân và các Mô hình ngôn ngữ lớn, cụ thể là khai thác sức mạnh của Google Gemini API trong việc xử lý ngôn ngữ tự nhiên.
- Nghiên cứu cơ chế hoạt động của kiến trúc Retrieval-Augmented Generation nhằm tối ưu hóa khả năng truy xuất thông tin và tổng hợp câu trả lời từ nguồn dữ liệu ngoài.
- Tìm hiểu và vận dụng các công cụ, thư viện mã nguồn mở hỗ trợ phát triển hệ thống AI như: ngôn ngữ lập trình Python, framework LangChain, cơ sở dữ liệu vector FAISS, và mô hình nhúng SentenceTransformers.

Phạm vi nghiên cứu:

- Phạm vi chức năng: Đề tài tập trung xây dựng một hệ thống Chatbot hỏi đáp (AI PDF Assistant) được tích hợp trên nền tảng Web với giao diện tương tác trực quan. Hệ thống cho phép người dùng tải lên các tệp tài liệu dưới nhiều định dạng phổ biến như PDF, DOCX, PPTX. Chatbot sẽ tiếp nhận câu hỏi bằng văn bản và sinh ra câu trả lời dựa trên chính dữ liệu nội dung từ các tài liệu đã được cung cấp, qua đó hạn chế tối đa việc AI tự bịa đặt thông tin.
- Phạm vi công nghệ: Hệ thống được phát triển chủ yếu dựa trên ngôn ngữ lập trình Python. Giao diện Web được xây dựng thông qua Flask và Streamlit. Quá trình tìm kiếm, lưu trữ và xử lý ngữ nghĩa tài liệu được thực hiện bởi FAISS,

LangChain, SentenceTransformers và Google Gemini API .

- Giới hạn đề tài: Do được giới hạn trong khuôn khổ của học phần Trí tuệ nhân tạo, đề tài chủ yếu tập trung vào việc hiện thực hóa và chứng minh tính khả thi của kiến trúc RAG trong xử lý văn bản, chưa đi sâu vào việc xử lý các dữ liệu đa phương tiện phức tạp (hình ảnh, biểu đồ phức tạp trong PDF) hay tối ưu hóa khả năng chịu tải của hệ thống trên quy mô lớn.

#### **4. Cấu trúc của báo cáo**

Chương 1. Lý thuyết cơ sở: Trình bày nền tảng về AI/NLP bao gồm Mô hình ngôn ngữ lớn, Google Gemini API, kỹ thuật Embedding và Vector Database. Tập trung làm rõ kiến trúc RAG và framework LangChain.

Chương 2. Thiết kế chi tiết và các thành phần hệ thống: Mô tả kiến trúc dữ liệu, cách trích xuất tài liệu đa định dạng (PDF, DOCX, PPTX), cơ chế tìm kiếm tương đồng và sơ đồ hệ thống. Đồng thời đề xuất các giải pháp quản lý ngữ cảnh và kiểm soát, hạn chế hiện tượng bịa đặt thông tin.

Chương 3. Triển khai ứng dụng và thực nghiệm kết quả: Đi sâu vào cấu trúc mã nguồn, xây dựng giao diện Web (Python Flask, Streamlit/HTML/CSS). Tiến hành kiểm thử với tài liệu lớn, đánh giá độ chính xác và tối ưu hóa tốc độ phản hồi của Gemini API.

## **CHƯƠNG 1: LÝ THUYẾT CƠ SỞ**

### **1.1 Kiến thức cơ bản về xử lý ngôn ngữ tự nhiên và trí tuệ nhân tạo**

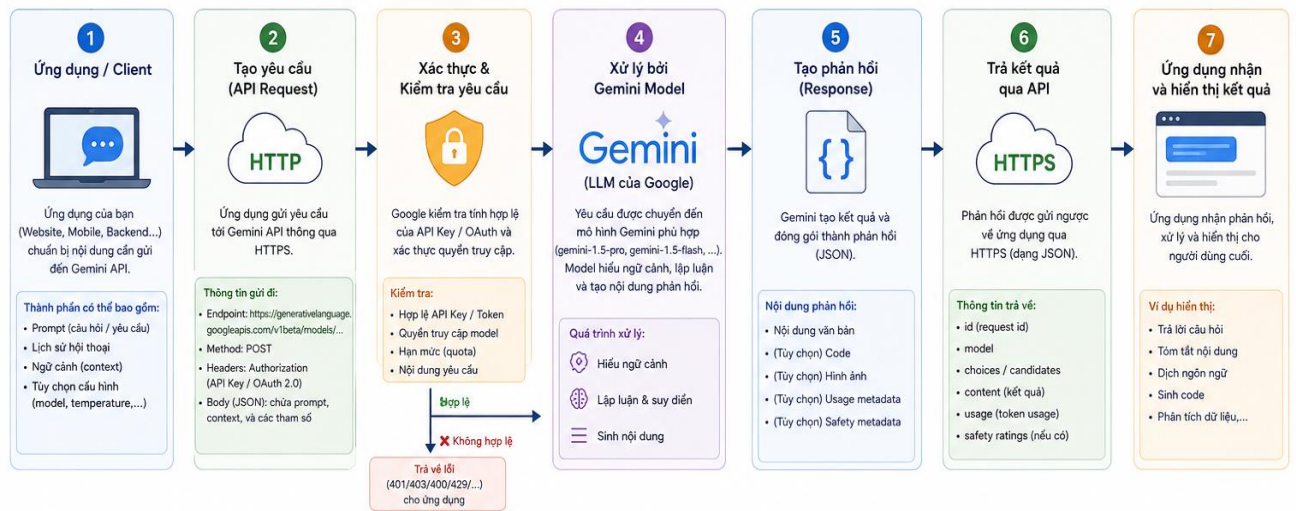
Trí tuệ nhân tạo đang trải qua giai đoạn phát triển bùng nổ, trong đó Xử lý ngôn ngữ tự nhiên đóng vai trò là cầu nối quan trọng giúp máy tính có thể hiểu, diễn giải và tạo ra ngôn ngữ của con người. NLP là sự kết hợp sâu sắc giữa ngôn ngữ học máy tính, khoa học máy tính và các mô hình học máy nhằm giải quyết các bài toán phức tạp như dịch máy, phân tích cảm xúc, tóm tắt văn bản và đặc biệt là hỏi đáp tự động.

#### **1.1.1 Khái niệm về mô hình ngữ lớn**

Mô hình ngôn ngữ lớn là một thể hệ mô hình NLP tiên tiến, được huấn luyện trên các tập dữ liệu văn bản khổng lồ với hàng tỷ, thậm chí hàng nghìn tỷ tham số. Hầu hết các LLM hiện nay đều dựa trên kiến trúc mạng nơ-ron Transformer, cho phép mô hình nắm bắt được ngữ cảnh và mối quan hệ phức tạp giữa các từ trong những đoạn văn bản dài một cách song song và hiệu quả. Nhờ kiến trúc này, LLM có khả năng suy luận, dịch thuật, viết mã lập trình và trả lời câu hỏi với độ trôi chảy gần giống con người. Tuy nhiên, các mô hình này vẫn gặp phải hạn chế cốt lõi về việc cập nhật thông tin thực tế theo thời gian thực và đôi khi có xu hướng sinh ra các thông tin hoàn toàn sai lệch nhưng lại mang văn phong rất thuyết phục.

#### **1.1.2 Tổng quan về mô hình Google Gemini API**

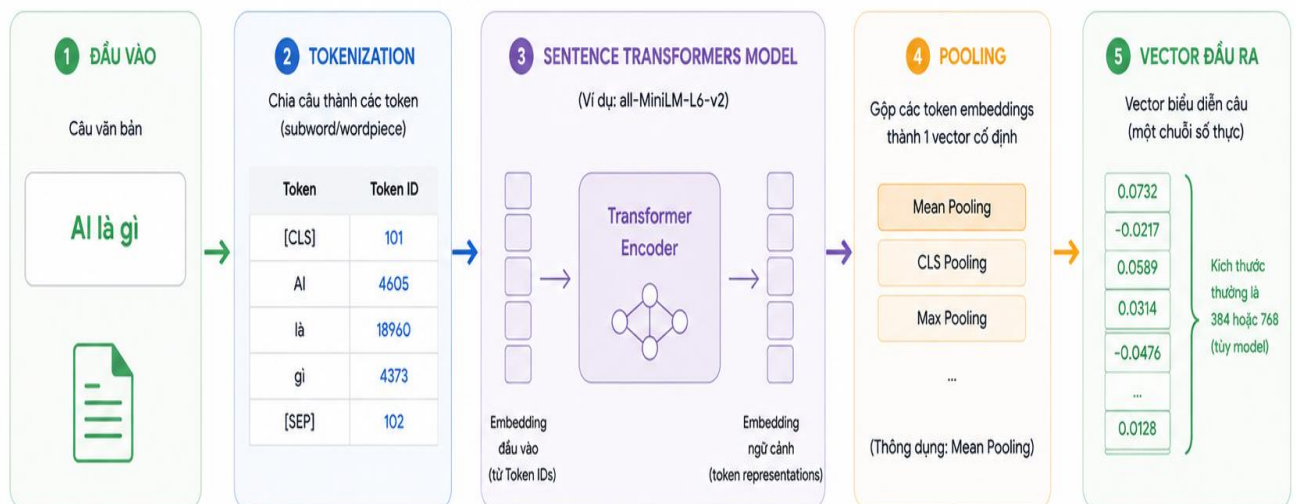
Google Gemini là một trong những mô hình ngôn ngữ tiên tiến và mạnh mẽ nhất hiện nay do Google DeepMind nghiên cứu và phát triển. Điểm khác biệt cốt lõi của Gemini so với các thể hệ mô hình trước là tính đa phương thức tự nhiên; nghĩa là nó được xây dựng ngay từ đầu để hiểu, phân tích và vận hành liền mạch trên nhiều loại dữ liệu khác nhau bao gồm văn bản, hình ảnh, âm thanh và mã nguồn lập trình. Việc Google cung cấp mô hình này thông qua giao diện lập trình ứng dụng cho phép các nhà phát triển dễ dàng tích hợp sức mạnh suy luận vượt trội của nó vào các phần mềm tùy chỉnh. Điều này đặc biệt hữu ích trong việc xây dựng các hệ thống Chatbot hoặc trợ lý ảo thông minh cần khả năng xử lý ngữ cảnh đa dạng và sâu sắc.



Hình 1.1 Sơ đồ hoạt động Google Gemini API

### 1.1.3 Kỹ thuật nhúng văn bản và SentenceTransformers

Để máy tính có thể xử lý và so sánh được ý nghĩa ngữ nghĩa của ngôn ngữ tự nhiên, các hệ thống AI cần phải chuyển đổi văn bản thành dữ liệu tính toán được thông qua kỹ thuật nhúng văn bản. Kỹ thuật này biểu diễn các từ, câu hoặc đoạn văn bản dưới dạng các vector số học trong một không gian toán học nhiều chiều. Đặc tính quan trọng nhất của không gian này là các đoạn văn bản có ý nghĩa tương đồng nhau sẽ được biểu diễn bằng các vector có khoảng cách hình học gần nhau. Nhằm phục vụ quá trình này, SentenceTransformers nổi lên như một thư viện Python chuyên dụng và mạnh mẽ, cung cấp các mô hình đã được huấn luyện sẵn để tạo ra các vector nhúng chất lượng cao. Thư viện này là công cụ đắc lực giúp tối ưu hóa quá trình so sánh độ tương đồng ngữ nghĩa giữa truy vấn của người dùng và các tài liệu tham khảo



Hình 1.2 Quá trình chuyển đổi văn bản thành Vecto

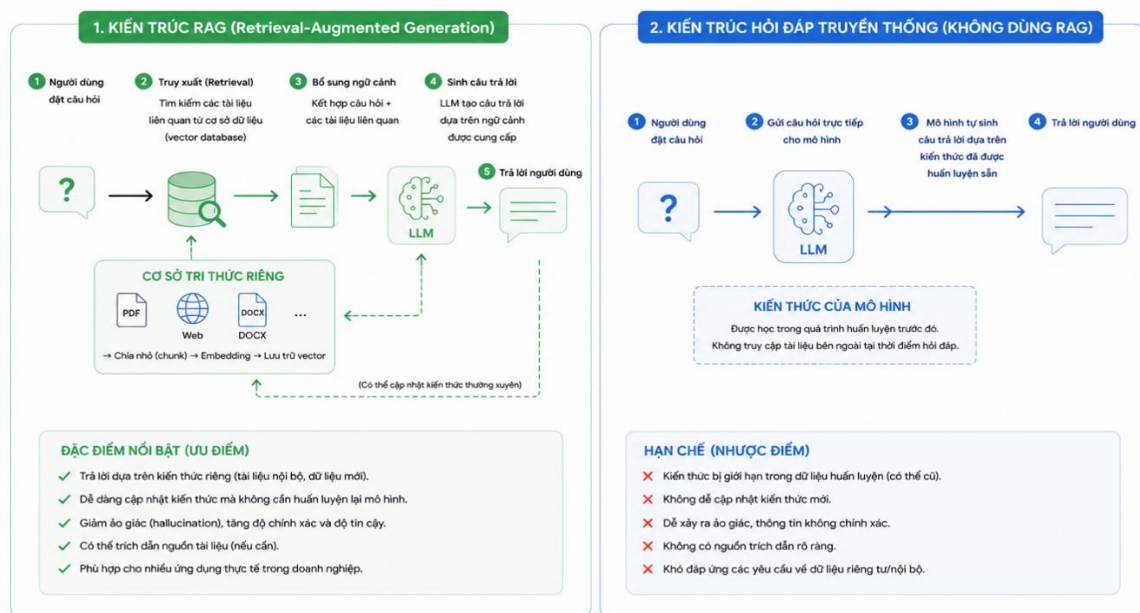
### 1.1.4 Khái niệm cơ bản về cơ sở dữ liệu Vector

Cơ sở dữ liệu Vector là một hệ thống lưu trữ thể hệ mới được thiết kế đặc thù để quản lý, lập chỉ mục và truy vấn các vector nhưng được tạo ra từ dữ liệu phi cấu trúc. Trái ngược với cơ sở dữ liệu quan hệ truyền thống thường chỉ thực hiện tìm kiếm dựa trên việc khớp chính xác các từ khóa, cơ sở dữ liệu Vector cho phép hệ thống thực hiện tìm kiếm theo độ tương đồng ngữ nghĩa. Bằng cách áp dụng các thuật toán tính toán khoảng cách toán học như Cosine Similarity hay L2 Distance, hệ thống có khả năng quét qua không gian lưu trữ đa chiều trong tích tắc để trích xuất ra những đoạn văn bản mang ý nghĩa sát nhất với câu hỏi đầu vào của người dùng từ một kho dữ liệu khổng lồ.

## 1.2 Tổng quan về kiến trúc RAG

### 1.2.1 RAG là gì và tại sao cần sử dụng RAG?

Retrieval-Augmented Generation là một phương pháp kết hợp đột phá giữa khả năng tìm kiếm, truy xuất thông tin từ các nguồn tri thức bên ngoài và khả năng sinh ngôn ngữ tự nhiên của mô hình ngôn ngữ lớn. Mặc dù các mô hình LLM sở hữu khả năng ngôn ngữ ưu việt, nhưng chúng luôn bị giới hạn bởi lượng kiến thức tĩnh tại thời điểm huấn luyện và hoàn toàn không thể truy cập vào các dữ liệu nội bộ, dữ liệu cá nhân hóa hay các luồng thông tin độc quyền của doanh nghiệp. Kiến trúc RAG ra đời để giải quyết triệt để rào cản này bằng cách chủ động cung cấp cho LLM những tài liệu nền tảng mang tính thực tế, cập nhật và liên quan nhất ngay trước khi mô hình tạo ra câu trả lời. Cơ chế này không chỉ nâng cao độ tin cậy, tính chính xác của câu trả lời mà còn giảm thiểu tối đa hiện tượng ảo giác thông tin.

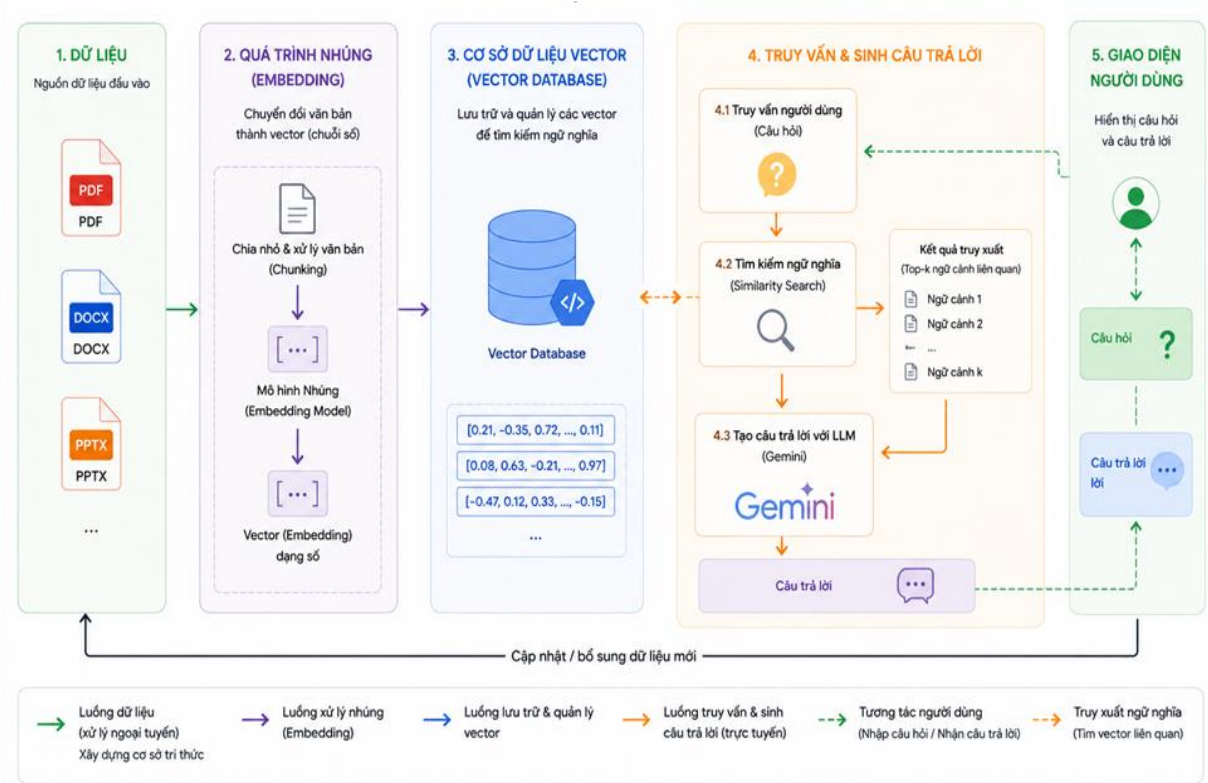


Hình 1.3 So sánh kiến trúc Rag và hệ thống truyền thống



### 1.2.2 Các khối chức năng cốt lõi trong kiến trúc RAG

Một hệ thống RAG cơ bản được cấu thành từ ba khối chức năng chính hoạt động chặt chẽ và liên kết với nhau. Khối đầu tiên là hệ thống tri thức, đóng vai trò như một kho lưu trữ tập trung các tài liệu đã được chuyển đổi thành định dạng vector. Khối thứ hai là thành phần Truy xuất, chịu trách nhiệm tiếp nhận câu hỏi của người dùng, biến đổi nó thành vector và đối chiếu để tìm ra các đoạn thông tin có mức độ liên quan cao nhất từ hệ thống tri thức. Khối cuối cùng là thành phần Sinh ngôn ngữ, thường được đảm nhận bởi một LLM mạnh mẽ, có nhiệm vụ tiếp nhận cả câu hỏi gốc và các thông tin vừa được truy xuất để tổng hợp, phân tích và sinh ra câu trả lời cuối cùng gửi đến người dùng.

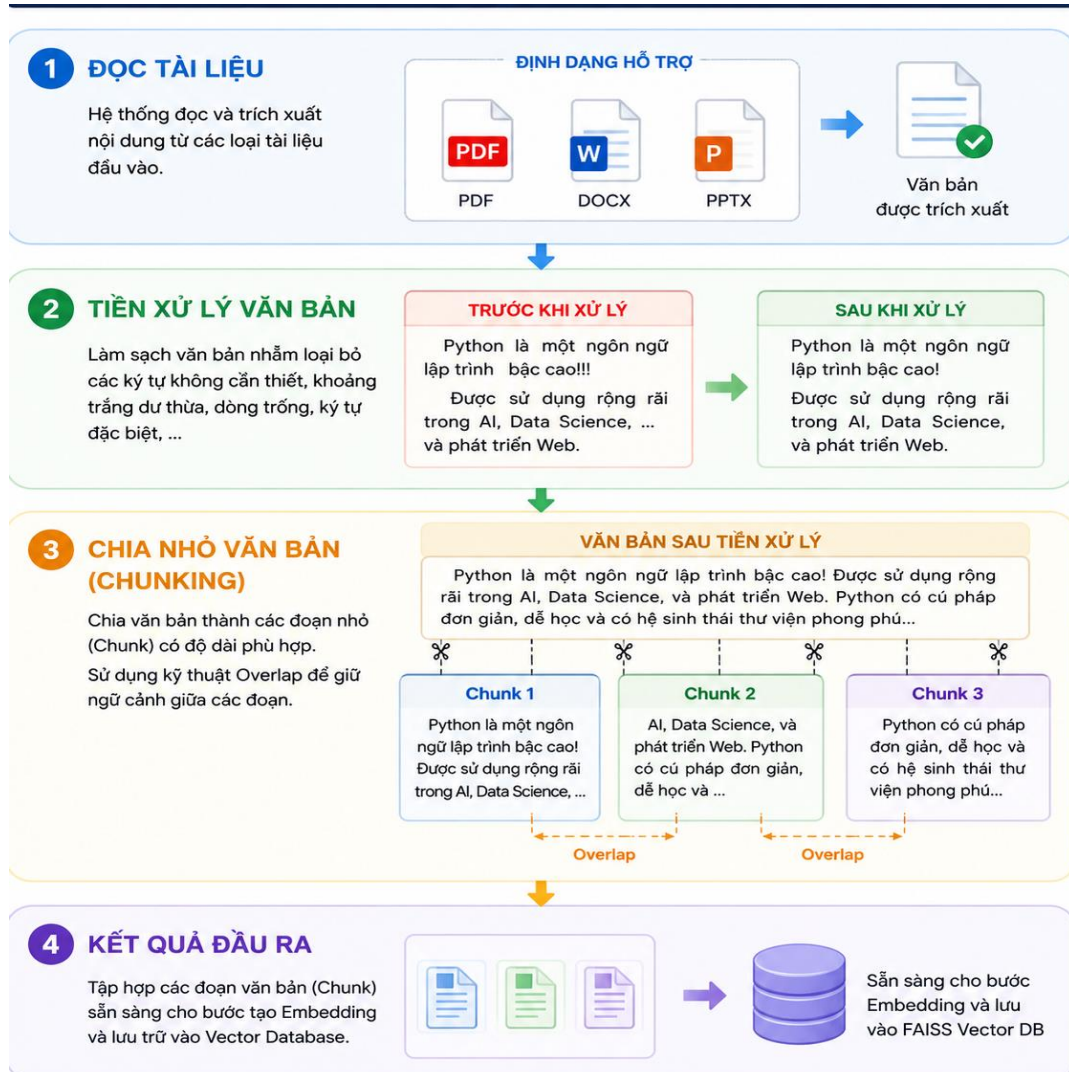


Hình 1.4 Sơ đồ tổng thể hệ thống RAG

### 1.2.3 Quy trình xử lý dữ liệu đầu vào

Trước khi dữ liệu có thể được truy xuất và sử dụng một cách hiệu quả, nó phải trải qua một quá trình tiền xử lý nghiêm ngặt. Hệ thống sẽ bắt đầu bằng việc đọc và trích xuất dữ liệu văn bản thô từ nhiều định dạng tập tin phổ biến khác nhau như PDF, DOCX hay bản trình bày PPTX. Sau đó, do các LLM có giới hạn nhất định về cửa sổ ngữ cảnh và để đảm bảo độ chính xác cao nhất cho hệ thống tìm kiếm vector, các tài liệu dài bắt buộc phải được chia nhỏ thành nhiều phần. Quá trình phân chia này nhằm tạo ra các đoạn văn bản có kích thước phù hợp, thông thường từ 500 đến 1000 tokens. Điều kiện tiên quyết của quá trình này là phải áp dụng các kỹ thuật chia tách thông

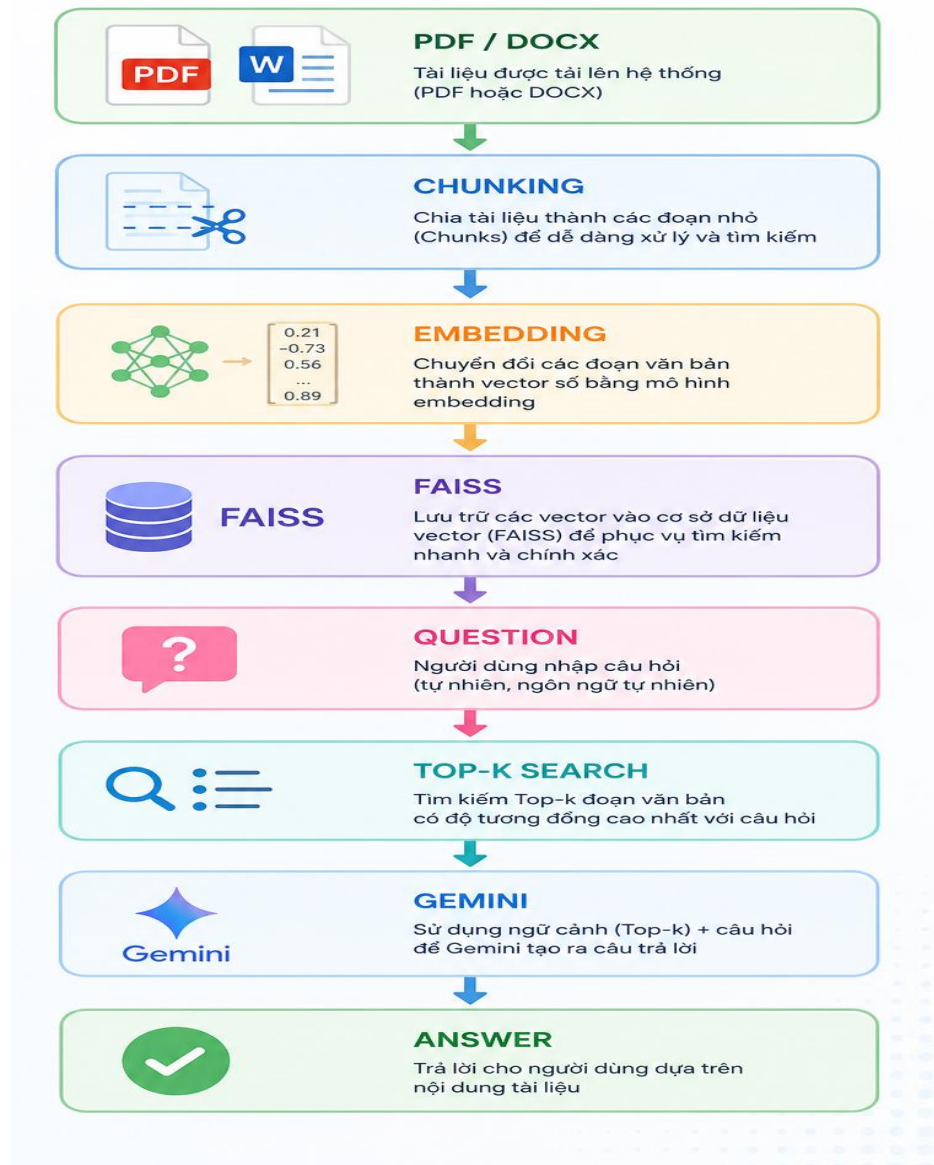
minh để đảm bảo ngữ cảnh không bị đứt gãy và giữ nguyên được mạch ý nghĩa trọn vẹn của văn bản gốc.



Hình 1.5 Sơ đồ quy trình tiền xử lý và chia nhỏ văn bản trong hệ thống

#### 1.2.4 Quy trình truy xuất dữ liệu và tạo câu trả lời

Khi hệ thống đi vào hoạt động thực tế, quy trình sẽ được kích hoạt ngay khi người dùng đưa ra một câu hỏi. Bước đầu tiên, hệ thống sử dụng mô hình Embedding để ánh xạ câu hỏi đó thành một vector số học. Trong pha truy xuất, vector đại diện cho câu hỏi này sẽ được đưa vào cơ sở dữ liệu Vector để truy vấn và thu về top các đoạn văn bản có độ tương đồng cao nhất. Bước sang pha sinh ngôn ngữ, các đoạn văn bản vừa được trích xuất này sẽ được ghép chung với câu hỏi gốc theo một cấu trúc mẫu để tạo thành một câu lệnh hoàn chỉnh và giàu ngữ cảnh. Câu lệnh này sau đó được chuyển đến LLM để mô hình tiến hành tổng hợp thông tin, chắt lọc nội dung và sinh ra câu trả lời cuối cùng một cách trôi chảy, logic và bám sát tuyệt đối vào nguồn tài liệu tham khảo.



Hình 1.6 Quy trình hoạt động của Chatbot RAG

### 1.2.5 Vai trò của FAISS trong hệ thống RAG

Trong kiến trúc RAG, FAISS đóng vai trò lưu trữ và tìm kiếm các vector embedding của tài liệu nhằm hỗ trợ quá trình truy xuất thông tin. Sau khi tài liệu được chia nhỏ và chuyển thành vector bằng Sentence Transformers, các vector này sẽ được lưu trong FAISS Index. Khi người dùng đặt câu hỏi, hệ thống chuyển câu hỏi thành vector và sử dụng FAISS để tìm ra các đoạn văn bản có nội dung liên quan nhất. Những đoạn văn bản này được cung cấp cho mô hình Gemini để sinh câu trả lời chính xác. Việc sử dụng FAISS giúp tăng tốc độ tìm kiếm, xử lý hiệu quả dữ liệu lớn và hạn chế hiện tượng sinh thông tin sai lệch.

### 1.2.6 Vai trò của Framework LangChain trong hệ thống RAG

Trong cấu trúc của hệ thống chatbot RAG, LangChain đóng vai trò là một framework nền tảng, giữ nhiệm vụ làm cầu nối công nghệ để tích hợp và kết nối chặt chẽ giữa mô hình ngôn ngữ lớn (Google Gemini), cơ sở dữ liệu vector (FAISS) cùng các nguồn dữ liệu bên ngoài. Không chỉ dừng lại ở việc kết nối, LangChain còn trực tiếp điều phối toàn bộ quy trình xử lý dữ liệu của hệ thống. Framework này tự động hóa vòng đời dữ liệu một cách toàn diện, từ các bước tiền xử lý như chia nhỏ văn bản, tạo vector embedding, cho đến việc truy xuất thông tin chính xác và chuyển giao ngữ cảnh để mô hình sinh câu trả lời. Nhờ cơ chế xây dựng các chuỗi xử lý thông minh, LangChain giúp tối ưu hóa tiến trình phát triển phần mềm bằng cách giảm thiểu độ phức tạp của mã nguồn, nâng cao khả năng tái sử dụng cấu trúc và đảm bảo tính linh hoạt. Qua đó, hệ thống không chỉ đạt hiệu quả cao trong vận hành mà còn dễ dàng bảo trì và mở rộng quy mô trong tương lai.

## 1.3 Tổng quan về Flask Framework

Flask là một micro web framework mã nguồn mở được phát triển bằng Python, cho phép xây dựng ứng dụng web một cách đơn giản và linh hoạt. Trong đề tài, Flask được sử dụng để xây dựng giao diện web và làm cầu nối giữa người dùng với các thành phần xử lý như FAISS và Gemini API, giúp hệ thống chatbot hoạt động trên môi trường web.

### 1.3.1 Giới thiệu Flask

Flask là một framework nhẹ được xây dựng trên ngôn ngữ Python. Flask cung cấp các chức năng cơ bản như định tuyến URL, xử lý yêu cầu HTTP và quản lý giao diện thông qua template HTML. Nhờ khả năng tích hợp tốt với các thư viện AI và Machine Learning, Flask là lựa chọn phù hợp để phát triển hệ thống chatbot hỏi đáp tài liệu trong đề tài này.

### 1.3.2 Mô hình Client – Server trong ứng dụng Web

Mô hình Client – Server là kiến trúc trong đó Client gửi yêu cầu đến Server, sau đó Server xử lý dữ liệu và trả kết quả về cho Client. Trong hệ thống chatbot RAG, người dùng nhập câu hỏi trên giao diện web, Flask Server tiếp nhận yêu cầu, thực hiện truy xuất dữ liệu liên quan từ FAISS, gửi ngữ cảnh đến Gemini API để tạo câu trả lời và trả kết quả về giao diện cho người dùng. Mô hình này giúp tách biệt phần giao diện và phần xử lý nghiệp vụ, đồng thời tăng khả năng mở rộng và bảo trì hệ thống.

## 1.4 Kết luận chương 1

Chương 1 đã trình bày các cơ sở lý thuyết làm nền tảng cho việc xây dựng hệ thống chatbot hỏi đáp tài liệu dựa trên kiến trúc RAG. Nội dung chương đã giới thiệu các khái niệm liên quan đến trí tuệ nhân tạo, xử lý ngôn ngữ tự nhiên, mô hình ngôn

ngữ lớn và Google Gemini API. Bên cạnh đó, chương cũng trình bày kỹ thuật nhúng văn bản bằng SentenceTransformers, cơ sở dữ liệu vector và vai trò của FAISS trong việc tìm kiếm ngữ nghĩa. Ngoài ra, kiến trúc Retrieval-Augmented Generation đã được phân tích nhằm làm rõ quy trình truy xuất thông tin và sinh câu trả lời dựa trên nội dung tài liệu. Chương cũng giới thiệu Flask Framework và mô hình Client – Server, là nền tảng để triển khai hệ thống chatbot trên môi trường web. Những kiến thức và công nghệ được trình bày trong chương này là cơ sở quan trọng cho quá trình phân tích, thiết kế và xây dựng hệ thống chatbot RAG được trình bày trong Chương 2.

## CHƯƠNG 2: THIẾT KẾ CHI TIẾT VÀ CÁC THÀNH PHẦN HỆ THỐNG CHATBOT RAG

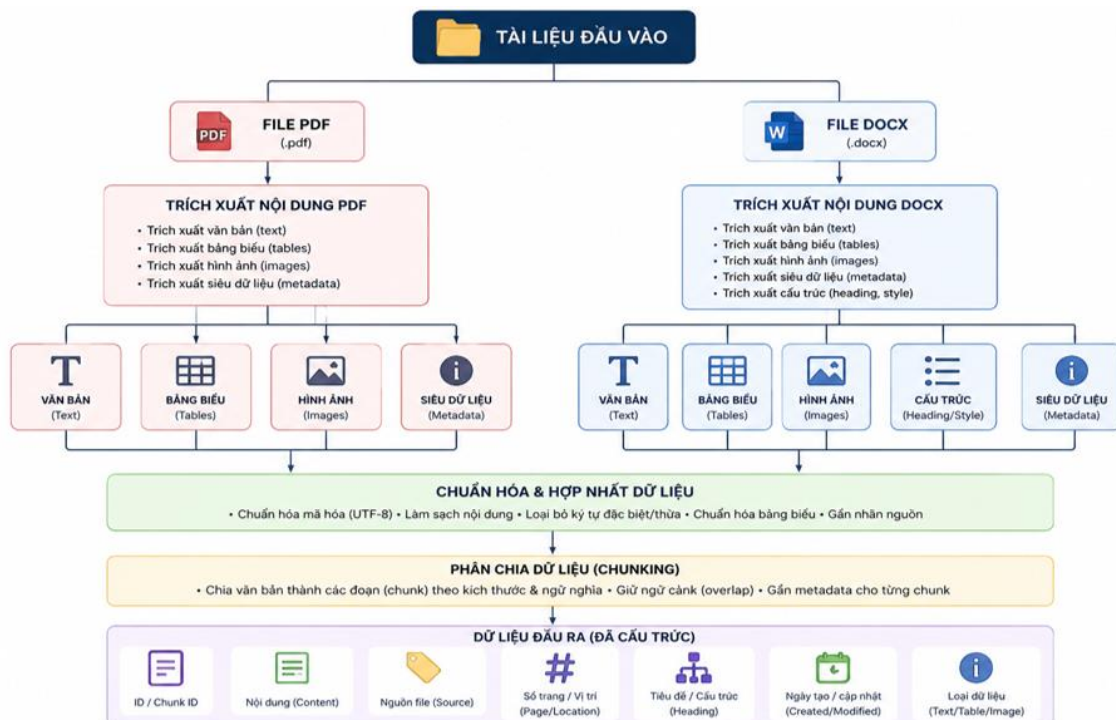
### 2.1 Kiến trúc dữ liệu và thành phần tìm kiếm vector FAISS

#### 2.1.1 Mô hình phân cấp cấu trúc tài liệu đầu vào (PDF, DOCX, PPTX)

Trong hệ thống Chatbot RAG, dữ liệu đầu vào đóng vai trò nền tảng quyết định trực tiếp đến độ chính xác và chất lượng phản hồi của mô hình. Các tài liệu được sử dụng trong hệ thống thường tồn tại ở nhiều định dạng khác nhau như PDF, DOCX và PPTX, đây là những định dạng phổ biến trong môi trường học thuật và doanh nghiệp. Để đảm bảo khả năng xử lý hiệu quả và tối ưu hóa quá trình truy xuất thông tin, hệ thống áp dụng mô hình phân cấp dữ liệu theo nhiều cấp độ. Cụ thể, dữ liệu được tổ chức theo ba lớp chính:

- Lớp tài liệu: Cấp cao nhất, gồm toàn bộ file gốc dưới dạng một đơn vị tri thức hoàn chỉnh.
- Lớp chương/mục: Chia nhỏ tài liệu theo tiêu đề hoặc cấu trúc logic nhằm duy trì ngữ nghĩa theo từng chủ đề.
- Lớp đoạn văn: Đơn vị nhỏ nhất, được chia theo số lượng token hoặc câu để tối ưu hóa cho mô hình embedding.

Việc phân cấp dữ liệu như vậy giúp hệ thống vừa giữ được cấu trúc ngữ nghĩa của tài liệu, vừa đảm bảo khả năng truy xuất chính xác theo từng phần nội dung nhỏ. Đồng thời, mô hình này giúp giảm nhiễu thông tin trong quá trình tìm kiếm vector.



Hình 2.1 Mô hình phân cấp dữ liệu



### 2.1.2 Cơ chế trích xuất nội dung văn bản đa định dạng

Hệ thống RAG cần xử lý đồng thời nhiều loại tài liệu khác nhau, do đó việc xây dựng cơ chế trích xuất dữ liệu đa định dạng là yêu cầu bắt buộc. Mỗi định dạng tài liệu có đặc điểm riêng, vì vậy hệ thống sử dụng các thư viện chuyên dụng để đảm bảo khả năng trích xuất chính xác.

- Với PDF: Dùng PyMuPDF hoặc pdfminer.six để trích xuất văn bản từ cấu trúc phức tạp, chia cột.
- Với DOCX: Dùng python-docx để đọc nội dung theo từng đoạn văn logic.
- Với PPTX: Dùng python-pptx để thu thập tiêu đề, bullet point và ghi chú trên từng slide.

Sau khi trích xuất, toàn bộ dữ liệu được chuẩn hóa về dạng văn bản thuần. Quá trình chuẩn hóa bao gồm: loại bỏ ký tự đặc biệt không cần thiết, xử lý xuống dòng và khoảng trắng dư thừa, chuẩn hóa mã hóa ký tự, loại bỏ các phần nội dung không mang giá trị ngữ nghĩa (header/footer, số trang,...). Tiếp theo, dữ liệu được chia nhỏ (chunking) theo chiến lược phù hợp. Việc chunking có thể dựa trên số lượng token cố định (ví dụ 300–500 tokens) hoặc theo câu, nhằm đảm bảo mỗi đoạn văn vẫn giữ được ngữ nghĩa đầy đủ nhưng không vượt quá giới hạn xử lý của mô hình embedding.

| THƯ VIỆN                                                                                                                                               | MỤC ĐÍCH                                                                                                                                                       | DÒNG CODE SỬ DỤNG TRONG FILE                                                                                                                         | ĐOẠN XỬ LÝ TRÍCH XUẤT VĂN BẢN                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1</b> Thư viện <b>pypdf</b><br><br>(file PDF)                    |  <b>Mục đích:</b><br>Dùng để trích xuất văn bản từ file PDF.                |  Khai báo ở đầu file:<br><pre>from pypdf import PdfReader</pre>   |  <b>Đoạn xử lý trích xuất văn bản:</b><br><pre>reader = PdfReader(file) for page in reader.pages:     page_text = page.extract_text() # &lt;--     Lệnh trích xuất chữ ở đây</pre>                                                                                |
| <b>2</b> Thư viện <b>python-docx</b><br><br>(file Word .docx)       |  <b>Mục đích:</b><br>Dùng để trích xuất văn bản từ file Word (.docx).       |  Khai báo ở đầu file:<br><pre>from docx import Document</pre>     |  <b>Đoạn xử lý trích xuất văn bản:</b><br><pre>doc = Document(file) for para in doc.paragraphs:     all_text += para.text + "\n" # &lt;--     Lệnh lấy chữ từ các đoạn văn</pre>                                                                                  |
| <b>3</b> Thư viện <b>python-pptx</b><br><br>(file PowerPoint .pptx) |  <b>Mục đích:</b><br>Dùng để trích xuất văn bản từ file PowerPoint (.pptx). |  Khai báo ở đầu file:<br><pre>from pptx import Presentation</pre> |  <b>Đoạn xử lý trích xuất văn bản:</b><br><pre>prs = Presentation(file) for slide in prs.slides:     for shape in slide.shapes:         if hasattr(shape, "text"):             all_text += shape.text + "\n" # &lt;--             Lệnh lấy chữ từ các slide</pre> |

Hình 2.2 Các thư viện Python dùng để trích xuất văn bản

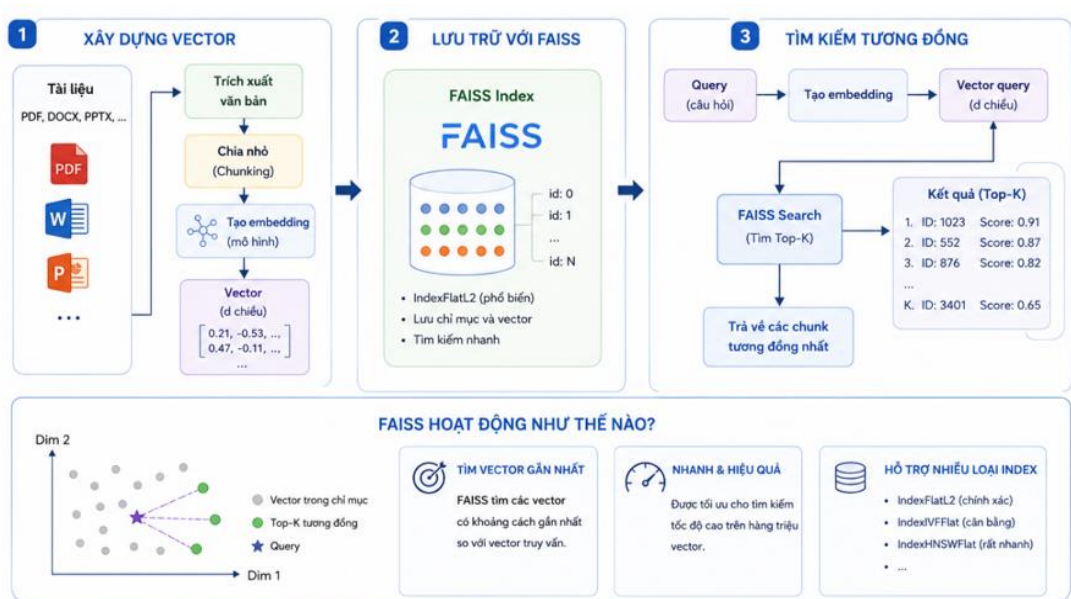
### 2.1.3 Phân lớp lưu trữ và tìm kiếm tương đồng với thư viện FAISS

Sau khi dữ liệu được xử lý và chia nhỏ thành các đoạn văn bản, hệ thống tiến hành chuyển đổi các đoạn này thành vector số học thông qua các mô hình embedding như Sentence-BERT hoặc OpenAI Embedding. Mỗi đoạn văn bản sẽ được biểu diễn dưới dạng một vector trong không gian nhiều chiều, trong đó các đoạn có nội dung tương tự sẽ nằm gần nhau trong không gian vector. Để lưu trữ và tìm kiếm hiệu quả trên tập dữ liệu lớn, hệ thống sử dụng thư viện FAISS (Facebook AI Similarity Search). Đây là một thư viện tối ưu hóa tìm kiếm vector do Facebook AI Research phát triển, có khả năng xử lý hàng triệu vector với tốc độ cao. FAISS hỗ trợ nhiều thuật toán tìm kiếm như:

- Flat Index: tìm kiếm chính xác nhưng chậm
- IVF (Inverted File Index): chia cụm để tăng tốc độ tìm kiếm
- HNSW (Hierarchical Navigable Small World): tìm kiếm tốc độ khá cao

Quy trình truy vấn diễn ra như sau:

1. Tạo Vector Query: Câu hỏi của người dùng được đưa vào mô hình để Tạo embedding, chuyển đổi thành một Vector query (d chiều).
2. FAISS Search (Tìm Top-K): Vector query này được đưa vào cấu trúc FAISS Search để so khớp, tìm các vector có khoảng cách gần nhất so với vector truy vấn trong kho chỉ mục.
3. Trả về kết quả (Top-K): Hệ thống tính toán điểm số tương đồng để lọc ra danh sách các Kết quả (Top-K) có điểm cao nhất, từ đó Trả về các chunk tương đồng nhất để làm ngữ cảnh sinh câu trả lời.



Hình 2.3 Cơ chế lưu trữ và tìm kiếm Faiss



## **2.2 Kiến trúc và chức năng của Chatbot AI Assistant**

### **2.2.1 Sơ đồ kiến trúc tổng thể của hệ thống Chatbot**

Hệ thống Chatbot RAG được thiết kế theo kiến trúc module hóa, bao gồm nhiều thành phần hoạt động phối hợp với nhau nhằm đảm bảo hiệu quả xử lý và khả năng mở rộng.

Các thành phần chính bao gồm:

- Giao diện người dùng: Tiếp nhận câu hỏi và hiển thị kết quả phản hồi (giao diện Web App/Chatbot UI).

- Bộ truy xuất thông tin: Sử dụng FAISS để tìm kiếm các đoạn văn bản liên quan nhất từ kho dữ liệu vector.

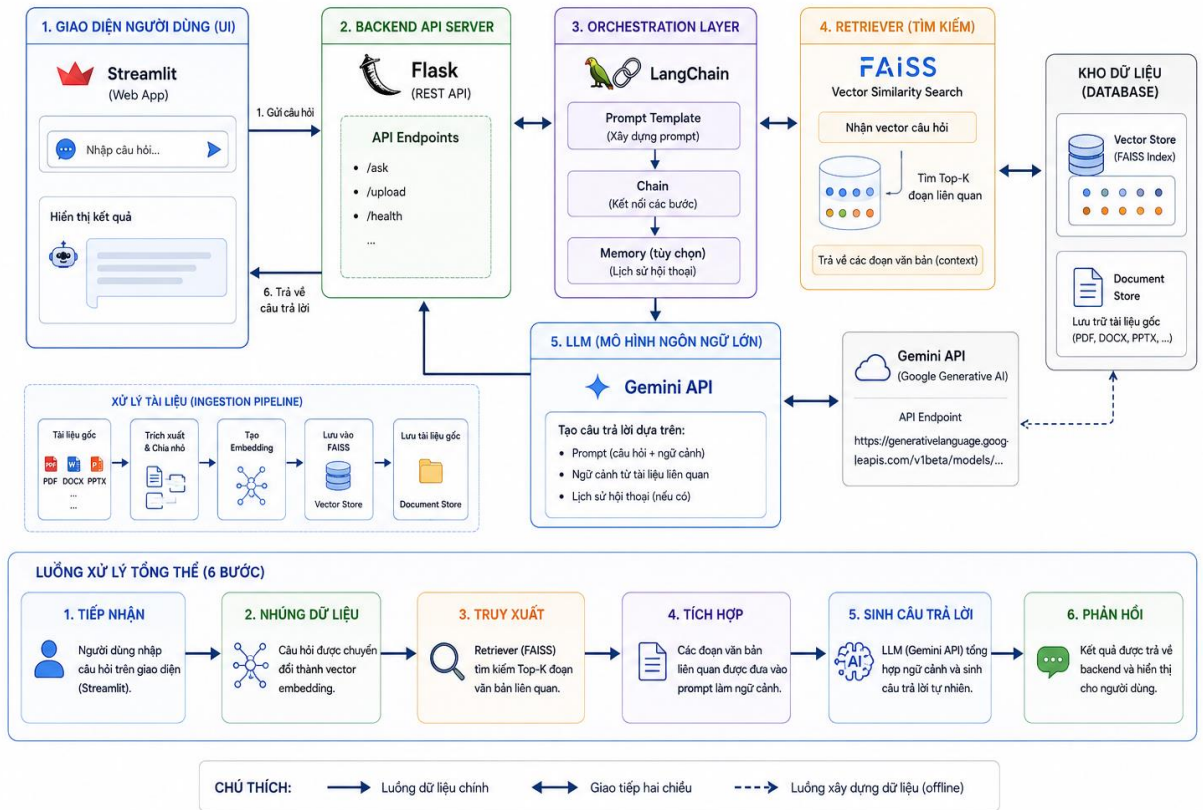
- Mô hình ngôn ngữ lớn: Chịu trách nhiệm sinh câu trả lời tự nhiên dựa trên ngữ cảnh được truy xuất.

- Kho dữ liệu: Lưu trữ tài liệu gốc và các vector embedding tương ứng.

Luồng xử lý tổng thể của hệ thống được mô tả như sau:

1. Tiếp nhận: Người dùng nhập câu hỏi vào hệ thống.
2. Nhúng dữ liệu: Câu hỏi được chuyển đổi thành vector embedding.
3. Truy xuất: Bộ Retriever tìm kiếm các đoạn dữ liệu liên quan trong kho FAISS.
4. Tích hợp: Các đoạn văn bản liên quan được đưa vào prompt làm ngữ cảnh cho LLM.
5. Sinh câu trả lời: LLM tổng hợp thông tin và tạo câu trả lời hoàn chỉnh.
6. Phản hồi: Kết quả cuối cùng được trả về và hiển thị trên giao diện người dùng.

Mô hình này giúp kết hợp giữa khả năng truy xuất thông tin chính xác và khả năng ngôn ngữ tự nhiên mạnh mẽ, tạo nên một hệ thống chatbot thông minh và đáng tin cậy.



Hình 2.4 Sơ đồ kiến trúc tổng thể hệ thống

### 2.2.2 Cơ chế truy xuất dữ liệu liên quan

Trong kiến trúc Retrieval-Augmented Generation, Retriever là thành phần đóng vai trò tìm kiếm và truy xuất các thông tin liên quan đến câu hỏi của người dùng từ kho tri thức của hệ thống. Mục tiêu của Retriever là cung cấp các ngữ cảnh phù hợp để hỗ trợ mô hình ngôn ngữ lớn tạo ra câu trả lời chính xác và bám sát nội dung tài liệu. Sau khi tài liệu PDF hoặc DOCX được xử lý, chia nhỏ thành các chunk và chuyển đổi thành vector embedding, toàn bộ các vector sẽ được lưu trữ trong chỉ mục FAISS. Khi người dùng nhập câu hỏi, hệ thống sẽ sử dụng cùng mô hình SentenceTransformers để chuyển câu hỏi thành một vector embedding có cùng không gian biểu diễn với dữ liệu đã lưu trữ. Tiếp theo, FAISS thực hiện quá trình tìm kiếm tương đồng ngữ nghĩa bằng cách so sánh vector của câu hỏi với các vector trong cơ sở dữ liệu. Dựa trên độ tương đồng giữa các vector, hệ thống sẽ lựa chọn ra một số đoạn văn bản có mức độ liên quan cao nhất theo phương pháp Top-k Search. Các đoạn văn bản được truy xuất sẽ được kết hợp thành ngữ cảnh (Context) và gửi đến mô hình Google Gemini API để sinh câu trả lời. Việc chỉ cung cấp các thông tin liên quan giúp giảm lượng dữ liệu đầu vào, tăng tốc độ xử lý và nâng cao chất lượng phản hồi của hệ thống. Quy trình truy xuất dữ liệu trong hệ thống được thực hiện theo các bước sau:

Bước 1: Người dùng nhập câu hỏi trên giao diện web.

Bước 2: Hệ thống chuyển câu hỏi thành vector embedding.

Bước 3: FAISS thực hiện tìm kiếm tương đồng ngữ nghĩa.

Bước 4: Lựa chọn Top-k đoạn văn bản có độ tương đồng cao nhất.

Bước 5: Trả về các đoạn văn bản liên quan làm ngữ cảnh cho Gemini.

Nhờ cơ chế Retriever, hệ thống chatbot có khả năng tìm kiếm thông tin theo ngữ nghĩa thay vì chỉ dựa trên từ khóa. Điều này giúp cải thiện đáng kể độ chính xác của câu trả lời, đặc biệt đối với các câu hỏi có cách diễn đạt khác nhau nhưng cùng mang một ý nghĩa.

### 2.2.3 Tích hợp Google Gemini API để sinh câu trả lời

Sau khi hoàn thành quá trình truy xuất dữ liệu liên quan từ kho tri thức, hệ thống tiến hành sử dụng Google Gemini API để tạo câu trả lời cho người dùng. Google Gemini là mô hình ngôn ngữ lớn do Google phát triển, có khả năng hiểu ngữ cảnh, xử lý ngôn ngữ tự nhiên và sinh văn bản với độ chính xác cao. Trong hệ thống chatbot RAG, Gemini không trực tiếp xử lý toàn bộ nội dung của tài liệu mà chỉ nhận các đoạn văn bản đã được Retriever lựa chọn. Các đoạn văn bản này được kết hợp với câu hỏi của người dùng để tạo thành ngữ cảnh đầu vào (Context). Việc cung cấp ngữ cảnh giúp mô hình tập trung vào những thông tin liên quan, từ đó tạo ra câu trả lời phù hợp với nội dung tài liệu. Quy trình sinh câu trả lời được thực hiện theo các bước sau:

Bước 1: Người dùng nhập câu hỏi trên giao diện web.

Bước 2: Hệ thống truy xuất các đoạn văn bản liên quan thông qua FAISS.

Bước 3: Kết hợp câu hỏi và các đoạn văn bản truy xuất thành một Prompt hoàn chỉnh.

Bước 4: Gửi Prompt đến Google Gemini API.

Bước 5: Gemini xử lý ngữ cảnh và sinh câu trả lời.

Bước 6: Kết quả được trả về Flask Server và hiển thị trên giao diện web.

Trong quá trình xây dựng hệ thống, Gemini API được tích hợp thông qua thư viện Python do Google cung cấp. Máy chủ Flask chịu trách nhiệm gửi yêu cầu đến API, nhận kết quả phản hồi và xử lý trước khi hiển thị cho người dùng. Việc sử dụng Google Gemini API mang lại nhiều lợi ích như khả năng hiểu ngôn ngữ tự nhiên tốt, tạo câu trả lời mạch lạc và dễ hiểu, đồng thời hỗ trợ xử lý đa dạng các dạng câu hỏi khác nhau. Khi kết hợp với kiến trúc RAG, Gemini có thể tạo ra các câu trả lời chính

xác hơn dựa trên thông tin được truy xuất từ tài liệu, góp phần nâng cao chất lượng và độ tin cậy của hệ thống chatbot.

#### **2.2.4 Chức năng kiểm soát thông tin và hạn chế hiện tượng “ảo tưởng”**

Một trong những vấn đề lớn của các mô hình ngôn ngữ hiện đại là hiện tượng “ảo giác”, tức là mô hình có thể tạo ra thông tin nghe có vẻ hợp lý nhưng không chính xác hoặc không tồn tại trong dữ liệu thực tế.

Hệ thống RAG được thiết kế để giảm thiểu vấn đề này bằng cách kết hợp cơ chế kiểm soát chặt chẽ nguồn dữ liệu:

- Chỉ sử dụng thông tin được truy xuất từ FAISS làm ngữ cảnh đầu vào
- Không cho phép mô hình trả lời hoàn toàn dựa trên kiến thức nội tại khi không có dữ liệu hỗ trợ
- Ưu tiên các đoạn văn có độ tương đồng cao nhất với câu hỏi
- Có thể thiết lập cơ chế “abstain”, tức là từ chối trả lời nếu không tìm thấy dữ liệu phù hợp

Ngoài ra, prompt cho mô hình cũng được thiết kế theo hướng:

- Yêu cầu chỉ trả lời dựa trên context được cung cấp
- Không tự suy diễn ngoài dữ liệu

Nhờ đó, hệ thống đảm bảo độ tin cậy cao hơn so với chatbot truyền thống sử dụng LLM đơn thuần.

### **2.3 Thiết kế giao diện Web Flask**

#### **2.3.1 Thiết kế giao diện người dùng**

Giao diện người dùng được xây dựng dưới dạng ứng dụng web chạy trên trình duyệt. Hệ thống bao gồm các thành phần chính như:

- Khu vực tải tài liệu: Cho phép người dùng tải lên các tệp PDF hoặc DOCX để hệ thống xử lý và xây dựng cơ sở tri thức.
- Khu vực nhập câu hỏi: Người dùng nhập nội dung câu hỏi cần tra cứu liên quan đến tài liệu đã tải lên.
- Khu vực hiển thị hội thoại: Hiển thị lịch sử trao đổi giữa người dùng và chatbot dưới dạng tin nhắn.
- Khu vực hiển thị kết quả: Trình bày câu trả lời do hệ thống tạo ra dựa trên nội dung tài liệu.

Giao diện được thiết kế theo mô hình một trang, giúp người dùng thực hiện toàn bộ thao tác trên cùng một màn hình mà không cần chuyển đổi giữa nhiều trang khác nhau. Điều này giúp tăng tính trực quan và nâng cao trải nghiệm sử dụng hệ thống.

### 2.3.2 Luồng tương tác người dùng và hệ thống

Quá trình tương tác giữa người dùng và hệ thống được thực hiện theo mô hình Client – Server. Người dùng thao tác trên trình duyệt web, trong khi Flask Server chịu trách nhiệm xử lý dữ liệu và trả kết quả. Luồng tương tác của hệ thống được thực hiện theo các bước sau:

- Bước 1: Người dùng truy cập giao diện web thông qua trình duyệt.
- Bước 2: Người dùng tải lên tài liệu PDF hoặc DOCX.
- Bước 3: Hệ thống tiếp nhận và xử lý tài liệu, thực hiện chia nhỏ văn bản, tạo embedding và lưu trữ vào FAISS.
- Bước 4: Người dùng nhập câu hỏi trên giao diện chatbot.
- Bước 5: Flask Server tiếp nhận câu hỏi và thực hiện truy xuất các đoạn văn bản liên quan từ FAISS.
- Bước 6: Các đoạn văn bản truy xuất được gửi cùng câu hỏi đến Google Gemini API để sinh câu trả lời.
- Bước 7: Kết quả được trả về giao diện web và hiển thị cho người dùng.

Quy trình tương tác trên giúp đảm bảo hệ thống hoạt động liên tục, phản hồi nhanh và cung cấp các câu trả lời phù hợp với nội dung tài liệu mà người dùng đã cung cấp.

## 2.4 Kết luận chương 2

Trong chương này, hệ thống Chatbot RAG đã được trình bày một cách chi tiết từ kiến trúc dữ liệu, cơ chế xử lý tài liệu đa định dạng cho đến kiến trúc tổng thể của hệ thống chatbot. Việc kết hợp giữa kỹ thuật embedding, tìm kiếm vector bằng FAISS và mô hình ngôn ngữ lớn giúp hệ thống đạt được sự cân bằng giữa độ chính xác thông tin và khả năng sinh ngôn ngữ tự nhiên. Đồng thời, các cơ chế như quản lý ngữ cảnh hội thoại và kiểm soát hallucination góp phần nâng cao tính ổn định, độ tin cậy và tính ứng dụng thực tế của hệ thống trong các bài toán hỏi đáp thông minh.

### CHƯƠNG 3: TRIỂN KHAI ỨNG DỤNG VÀ THỰC NGHIỆM KẾT QUẢ

Chương này sẽ tập trung trình bày quá trình hiện thực hóa hệ thống Chatbot hỏi đáp tài liệu. Các nội dung chính bao gồm việc xây dựng ứng dụng web, ứng dụng công nghệ lõi và đánh giá các kết quả đạt được sau khi triển khai.

#### 3.1 Môi trường phát triển và công nghệ sử dụng

Để xây dựng hệ thống chatbot hỏi đáp tài liệu dựa trên kiến trúc RAG, đề tài sử dụng ngôn ngữ lập trình Python cùng với nhiều thư viện và công nghệ hỗ trợ khác. Việc lựa chọn các công nghệ này nhằm đảm bảo hệ thống có khả năng xử lý dữ liệu hiệu quả, dễ triển khai và có khả năng mở rộng trong tương lai. Các công nghệ chính được sử dụng trong hệ thống bao gồm:

- Python: Là ngôn ngữ lập trình chính được sử dụng để xây dựng toàn bộ hệ thống. Python có cú pháp đơn giản, dễ phát triển và hỗ trợ mạnh mẽ cho các ứng dụng trí tuệ nhân tạo, xử lý ngôn ngữ tự nhiên và phát triển web.
- Flask Framework: Được sử dụng để xây dựng ứng dụng web, quản lý các yêu cầu từ người dùng, xử lý dữ liệu phía máy chủ và kết nối giữa giao diện web với các thành phần của hệ thống.
- Google Gemini API: Là mô hình ngôn ngữ lớn được sử dụng để sinh câu trả lời dựa trên ngữ cảnh được truy xuất từ tài liệu. Gemini hỗ trợ xử lý ngôn ngữ tự nhiên và tạo ra các câu trả lời mạch lạc, dễ hiểu.
- SentenceTransformers: Được sử dụng để chuyển đổi văn bản thành các vector embedding nhằm phục vụ cho quá trình tìm kiếm ngữ nghĩa.
- FAISS (Facebook AI Similarity Search): Là thư viện hỗ trợ lưu trữ và tìm kiếm các vector embedding với tốc độ cao. FAISS đóng vai trò quan trọng trong quá trình truy xuất các đoạn văn bản liên quan đến câu hỏi của người dùng.
- PyPDF2 và python-docx: Là các thư viện hỗ trợ đọc và trích xuất nội dung từ các tệp PDF và DOCX, giúp hệ thống xử lý nhiều định dạng tài liệu khác nhau.
- HTML, CSS và JavaScript: Được sử dụng để xây dựng giao diện người dùng trên nền tảng web, giúp tăng tính trực quan và nâng cao trải nghiệm sử dụng hệ thống.

*Bảng 3.1 Trình bày tóm tắt các công nghệ được sử dụng trong đề tài*

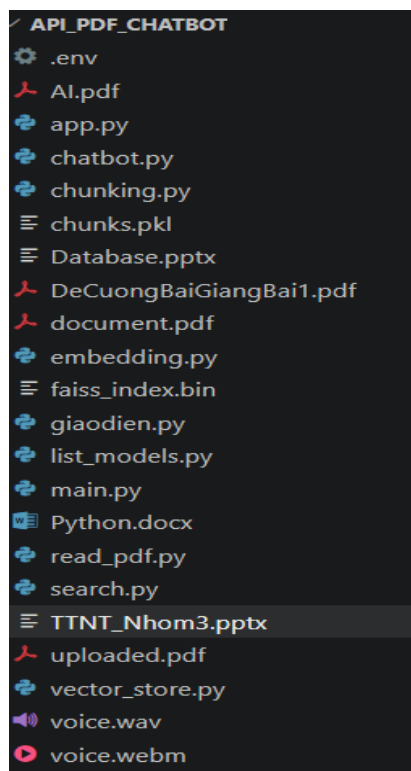
| STT | Công nghệ | Mục đích sử       |
|-----|-----------|-------------------|
| 1   | Python    | Xây dựng hệ thống |

|   |                      |                               |
|---|----------------------|-------------------------------|
| 2 | Flask                | Phát triển ứng dụng web       |
| 3 | Google Gemini API    | Sinh câu trả lời              |
| 4 | SentenceTransformers | Tạo vector embedding          |
| 5 | FAISS                | Lưu trữ và tìm kiếm vector    |
| 6 | PyPDF2, python-docx  | Đọc tài liệu PDF, DOCX        |
| 7 | HTML/CSS/JavaScript  | Xây dựng giao diện người dùng |

Việc kết hợp các công nghệ trên giúp hệ thống chatbot hoạt động ổn định, hỗ trợ xử lý hiệu quả dữ liệu tài liệu và cung cấp câu trả lời chính xác cho người dùng.

### 3.2 Cấu trúc mã nguồn và giải pháp công nghệ (Python Flask)

Hệ thống được phát triển chủ yếu bằng ngôn ngữ lập trình Python, kết hợp cùng các framework mạnh mẽ để xử lý cả backend lẫn frontend.



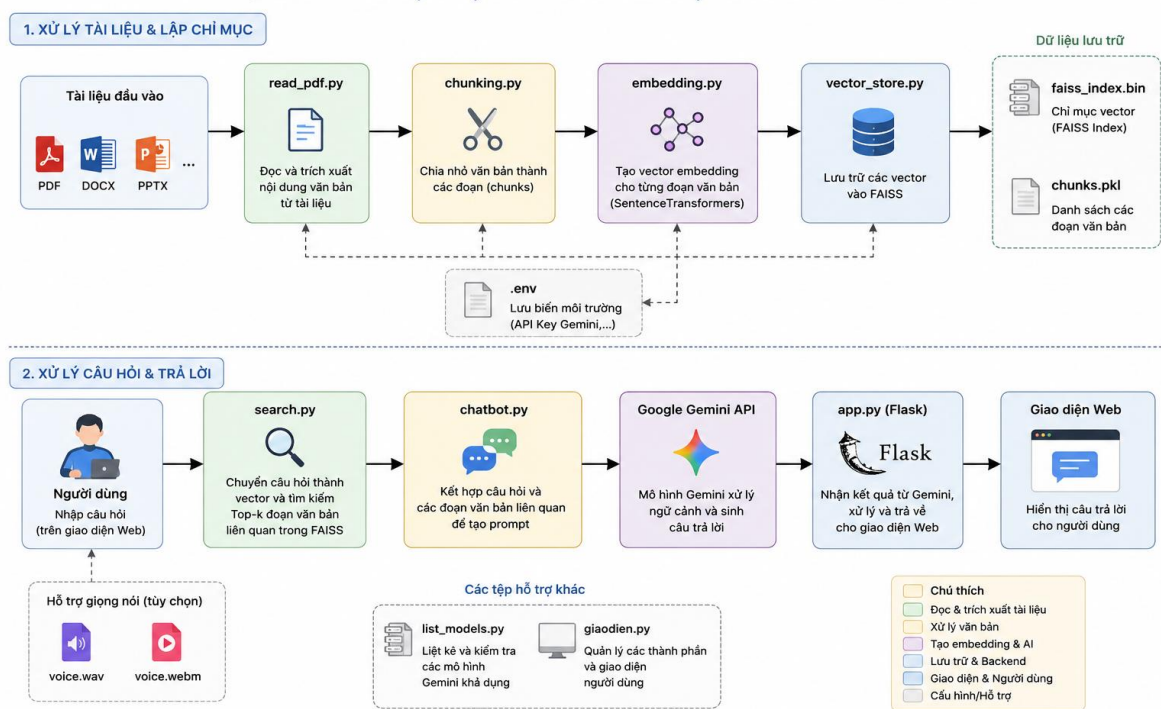
Hình 3.1 Cấu trúc mã nguồn

Bảng 3.2 Chức năng các tệp mã nguồn trong hệ thống

| STT | Tệp/Thư mục     | Chức năng                                                                                                                                      |
|-----|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | .env            | Lưu trữ các biến môi trường như khóa API Gemini nhằm đảm bảo tính bảo mật của hệ thống.                                                        |
| 2   | app.py          | Tệp chính của ứng dụng Flask, chịu trách nhiệm khởi chạy máy chủ web, tiếp nhận yêu cầu từ người dùng và điều phối các chức năng của hệ thống. |
| 3   | chatbot.py      | Chứa các hàm xử lý hội thoại, kết nối giữa bộ truy xuất dữ liệu và mô hình Gemini để sinh câu trả lời.                                         |
| 4   | chunking.py     | Thực hiện chức năng chia nhỏ nội dung tài liệu thành các đoạn văn bản (chunks) phục vụ quá trình tạo embedding và tìm kiếm ngữ nghĩa.          |
| 5   | embedding.py    | Sử dụng mô hình SentenceTransformers để chuyển đổi văn bản thành vector embedding.                                                             |
| 6   | vector_store.py | Quản lý việc lưu trữ và truy xuất dữ liệu vector trong FAISS.                                                                                  |
| 7   | search.py       | Thực hiện tìm kiếm ngữ nghĩa trên chỉ mục FAISS nhằm tìm ra các đoạn văn bản liên quan đến câu hỏi của người dùng                              |
| 8   | read_pdf.py     | Đọc và trích xuất nội dung văn bản từ các tệp PDF hoặc DOCX được tải lên.                                                                      |
| 9   | giaodien.py     | Chứa các thành phần liên quan đến xây dựng và quản lý giao diện người dùng của hệ thống.                                                       |



|    |                                                                                                    |                                                                                                 |
|----|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 10 | main.py                                                                                            | Tập điều phối hoặc chạy thử nghiệm các chức năng chính của hệ thống trong quá trình phát triển. |
| 11 | list_models.py                                                                                     | Liệt kê và kiểm tra các mô hình Gemini khả dụng trước khi tích hợp vào hệ thống.                |
| 12 | faiss_index.bin                                                                                    | Tập lưu trữ chỉ mục vector được tạo bởi FAISS, phục vụ quá trình truy xuất dữ liệu.             |
| 13 | chunks.pkl                                                                                         | Lưu trữ danh sách các đoạn văn bản sau khi thực hiện quá trình chunking.                        |
| 14 | uploaded.pdf,<br>document.pdf,<br>AI.pdf,<br>Python.docx,<br>Database.pptx,<br>TTNT_Nhom3.<br>pptx | Các tài liệu đầu vào được sử dụng để kiểm thử và đánh giá hệ thống                              |
| 15 | voice.wav,<br>voice.webm                                                                           | Các tệp âm thanh phục vụ chức năng nhận dạng và xử lý giọng nói của chatbot.                    |



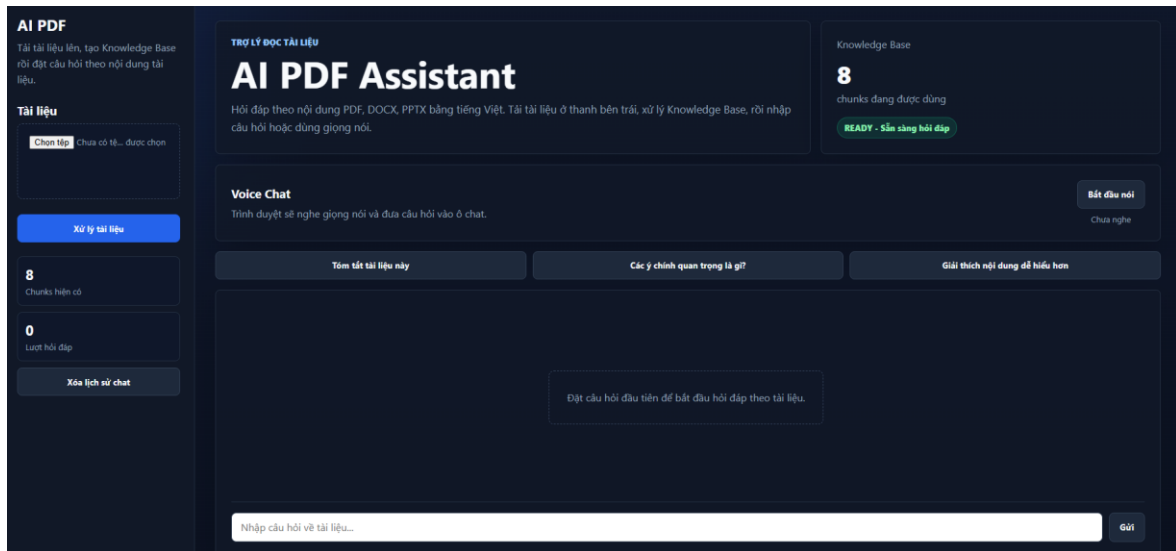
Hình 3.2 Quy trình hoạt động mã nguồn hệ thống Chatbot Rag

### 3.2.1 Thiết kế kiến trúc Backend ứng dụng với Python Flask

Framework Flask được sử dụng để xây dựng kiến trúc web application nền tảng. Backend sử dụng Flask chịu trách nhiệm xử lý các luồng dữ liệu phức tạp, bao gồm việc tiếp nhận file người dùng tải lên, gọi các hàm trích xuất dữ liệu đa định dạng (PDF, DOCX, PPTX). Bên cạnh đó, Flask còn đóng vai trò điều phối luồng dữ liệu thông qua framework LangChain, chuyển đổi văn bản thành vector bằng SentenceTransformers, lưu trữ vào hệ thống cơ sở dữ liệu vector FAISS và cuối cùng là giao tiếp với Google Gemini API để lấy kết quả.

### 3.2.2 Xây dựng giao diện Chatbot hiện đại trực quan với HTML

Để nâng cao trải nghiệm người dùng và tạo môi trường tương tác thuận tiện, hệ thống chatbot được xây dựng giao diện web bằng HTML kết hợp với CSS và JavaScript. Giao diện được thiết kế theo hướng hiện đại, trực quan và thân thiện, giúp người dùng dễ dàng thao tác trong quá trình sử dụng hệ thống. HTML được sử dụng để xây dựng cấu trúc chính của trang web, bao gồm các thành phần như khu vực tải tài liệu, cửa sổ hội thoại, ô nhập câu hỏi và khu vực hiển thị kết quả. CSS được sử dụng để định dạng giao diện, tạo bố cục hợp lý, màu sắc hài hòa và đảm bảo tính thẩm mỹ cho ứng dụng. Ngoài ra, JavaScript hỗ trợ xử lý các tương tác phía người dùng như gửi câu hỏi, hiển thị tin nhắn động và cập nhật nội dung mà không cần tải lại toàn bộ trang web.



Hình 3.3 Giao diện chính của hệ thống Chatbot

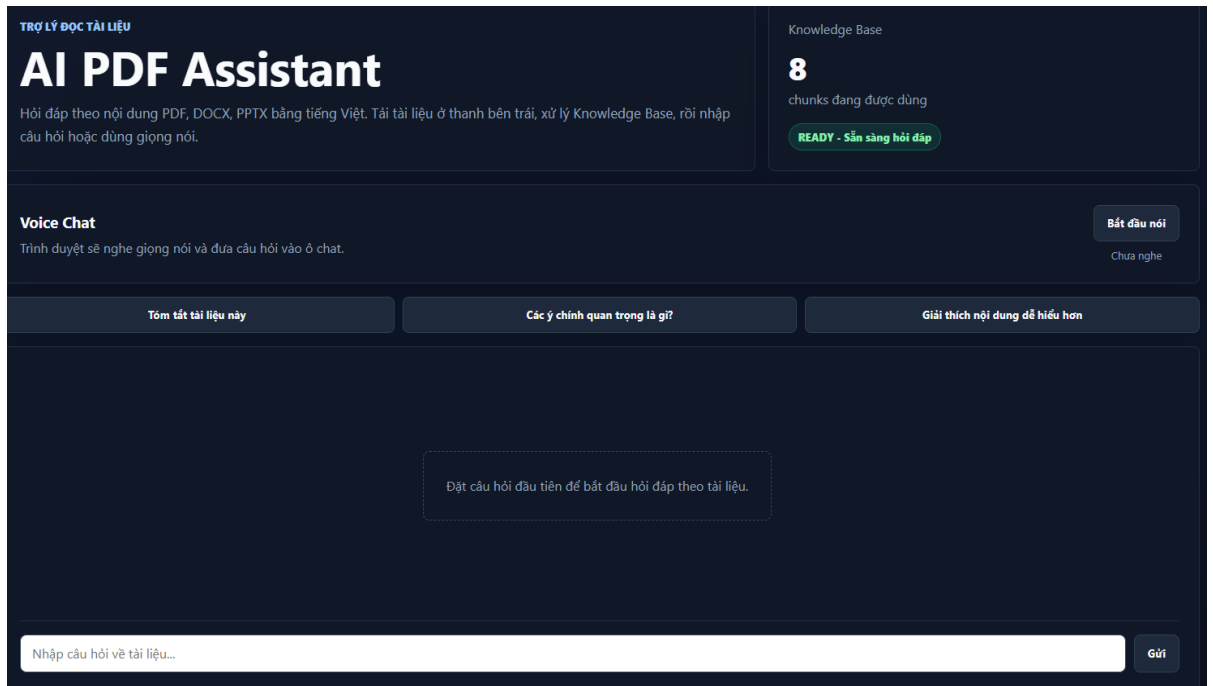
Giao diện chatbot được thiết kế với các chức năng chính như sau:

- Khu vực tải tài liệu: Cho phép người dùng tải lên các tệp PDF hoặc DOCX để xây dựng cơ sở tri thức cho hệ thống.



Hình 3.4 Chức năng tải tài liệu lên hệ thống

- Khu vực hội thoại: Hiển thị lịch sử trao đổi giữa người dùng và chatbot dưới dạng các khung tin nhắn, giúp người dùng dễ dàng theo dõi nội dung cuộc trò chuyện.



Hình 3.5 Giao diện khu vực hội thoại

- Ô nhập câu hỏi: Cho phép người dùng nhập câu hỏi liên quan đến nội dung tài liệu.
- Khu vực phản hồi: Hiển thị câu trả lời được tạo ra bởi hệ thống sau khi thực hiện quá trình truy xuất dữ liệu và sinh phản hồi bằng Gemini API.



Hình 3.6 Kết quả trả lời câu hỏi dựa trên nội dung tài liệu

Bên cạnh đó, giao diện còn được tối ưu để đảm bảo khả năng hiển thị tốt trên nhiều kích thước màn hình khác nhau, giúp người dùng có thể truy cập và sử dụng hệ thống một cách thuận tiện. Việc xây dựng giao diện chatbot bằng HTML, CSS và JavaScript kết hợp với Flask không chỉ giúp tăng tính trực quan của hệ thống mà còn góp phần cải thiện trải nghiệm người dùng, tạo điều kiện thuận lợi cho quá trình tương tác và khai thác thông tin từ tài liệu.

### **3.3 Xây dựng các chức năng chính**

Hệ thống chatbot được xây dựng dựa trên kiến trúc RAG nhằm hỗ trợ người dùng tra cứu thông tin từ các tài liệu PDF và DOCX. Để đảm bảo hệ thống hoạt động hiệu quả, đề tài triển khai các chức năng chính bao gồm xử lý tài liệu, tìm kiếm ngữ nghĩa, sinh câu trả lời và xây dựng giao diện tương tác trên nền tảng web.

#### **3.3.1 Xử lý và lập chỉ mục tài liệu**

Chức năng đầu tiên của hệ thống là tiếp nhận và xử lý tài liệu do người dùng tải lên. Hệ thống hỗ trợ hai định dạng tài liệu phổ biến là PDF và DOCX. Sau khi tài liệu được tải lên, hệ thống tiến hành trích xuất nội dung văn bản bằng các thư viện chuyên dụng. Nội dung sau khi được trích xuất sẽ được chia thành nhiều đoạn văn bản nhỏ nhằm đảm bảo hiệu quả trong quá trình tìm kiếm và truy xuất dữ liệu. Tiếp theo, mỗi chunk sẽ được chuyển đổi thành vector embedding thông qua mô hình SentenceTransformers. Các vector này được lưu trữ dưới dạng chỉ mục trong FAISS để phục vụ cho quá trình tìm kiếm ngữ nghĩa. Việc xây dựng chỉ mục giúp hệ thống có thể truy xuất dữ liệu nhanh chóng ngay cả khi xử lý các tài liệu có kích thước lớn.

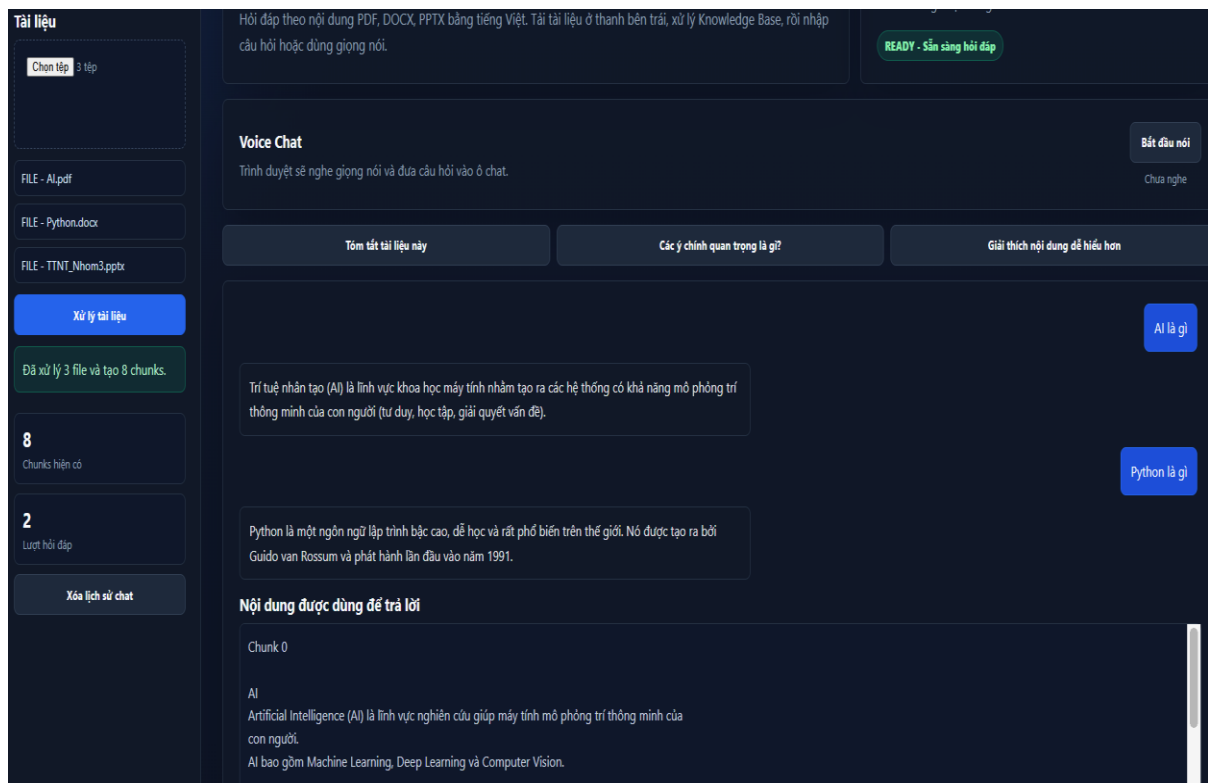
#### **3.3.2 Tìm kiếm ngữ nghĩa bằng FAISS**

Khi người dùng nhập câu hỏi, hệ thống sẽ chuyển câu hỏi thành vector embedding bằng cùng mô hình SentenceTransformers đã sử dụng trong quá trình xử lý tài liệu. Vector của câu hỏi sẽ được đưa vào FAISS để thực hiện tìm kiếm tương đồng ngữ nghĩa. FAISS sử dụng các phép đo khoảng cách giữa các vector để xác định những đoạn văn bản có nội dung liên quan nhất đến câu hỏi của người dùng. Hệ thống áp dụng phương pháp Top-k Search để lựa chọn một số đoạn văn bản có độ tương đồng cao nhất. Các đoạn văn bản này sẽ được sử dụng làm ngữ cảnh đầu vào cho mô hình Gemini nhằm sinh câu trả lời chính xác và bám sát nội dung tài liệu.

#### **3.3.3 Sinh câu trả lời bằng Gemini API**

Sau khi hoàn thành quá trình truy xuất dữ liệu, hệ thống kết hợp câu hỏi của người dùng với các đoạn văn bản liên quan để tạo thành một Prompt hoàn chỉnh. Prompt này được gửi đến Google Gemini API để xử lý và sinh câu trả lời. Gemini có khả năng hiểu ngữ cảnh và tạo văn bản tự nhiên, giúp hệ thống đưa ra các phản hồi mạch lạc và dễ hiểu. Việc kết hợp giữa cơ chế truy xuất dữ liệu của RAG và khả năng

sinh ngôn ngữ của Gemini giúp hạn chế hiện tượng tạo thông tin sai lệch, đồng thời nâng cao độ chính xác của câu trả lời.



Hình 3.7 Kết quả câu trả lời bằng Google API

### 3.3.4 Xây dựng giao diện Web với Flask

Để người dùng có thể tương tác với hệ thống một cách thuận tiện, đề tài sử dụng Flask Framework để xây dựng ứng dụng web. Flask đảm nhiệm vai trò tiếp nhận yêu cầu từ giao diện người dùng, xử lý dữ liệu phía máy chủ và trả kết quả về trình duyệt. Giao diện web được xây dựng bằng HTML, CSS và JavaScript nhằm tạo môi trường tương tác trực quan và thân thiện. Thông qua giao diện web, người dùng có thể tải tài liệu, đặt câu hỏi và xem kết quả trả lời ngay trên trình duyệt. Việc triển khai hệ thống trên nền tảng Flask giúp ứng dụng dễ dàng mở rộng, bảo trì và triển khai trong thực tế.

## 3.4 Các vấn đề kỹ thuật và đánh giá chất lượng hệ thống

### 3.4.1 Kịch bản kiểm thử ứng dụng với các file tài liệu lớn

Hệ thống đã được thử nghiệm với nhiều loại tài liệu thực tế, bao gồm việc hỗ trợ upload nhiều định dạng tài liệu như PDF, DOCX và PPTX. Quá trình kiểm thử cho thấy thư viện hỗ trợ tìm kiếm vector FAISS và mô hình SentenceTransformers hoạt động ổn định khi phân mảnh và truy xuất thông tin từ các tệp tài liệu có dung lượng vừa và lớn.

### 3.4.2 Đánh giá độ chính xác của câu trả lời dựa trên nội dung tài liệu

Nhờ việc hệ thống hoạt động theo kiến trúc RAG, độ chính xác khi truy xuất thông tin đã được tăng lên đáng kể. Điểm sáng lớn nhất của hệ thống là khả năng trả lời câu hỏi bám sát dựa trên nội dung tài liệu đã tải lên. Đặc biệt, cơ chế này giúp hệ thống hạn chế tối đa việc bịa thông tin, đảm bảo tính tin cậy cho người dùng khi tra cứu tài liệu học thuật hoặc chuyên ngành.

### 3.4.3 Tối ưu hóa tốc độ phản hồi và xử lý lỗi khi gọi Gemini API

Trong quá trình vận hành, việc kết nối với Google Gemini API đòi hỏi sự ổn định cao. Mã nguồn đã được tinh chỉnh để tối ưu hóa thời gian chờ khi API xử lý các prompt dài được gửi từ LangChain. Các cơ chế bắt lỗi cũng được triển khai để đảm bảo hệ thống web không bị sập khi API quá tải hoặc gặp sự cố mạng.

## 3.5 Kết quả đạt được

*Bảng 3.3 Kết quả đạt được*

| STT | Câu hỏi kiểm thử                     | Nội dung tài | Kết quả trả lời của hệ thống                                                                          | Đánh giá  |
|-----|--------------------------------------|--------------|-------------------------------------------------------------------------------------------------------|-----------|
| 1   | FAISS là gì?                         | AI.pdf       | Hệ thống trả lời đúng khái niệm FAISS là thư viện tìm kiếm vector do Facebook AI Research phát triển. | Chính xác |
| 2   | RAG là gì?                           | AI.pdf       | Hệ thống giải thích đúng kiến trúc Retrieval-Augmented Generation và vai trò của nó.                  | Chính xác |
| 3   | SentenceTransformers dùng để làm gì? | AI.pdf       | Hệ thống trả lời đúng chức năng tạo vector embedding cho văn bản.                                     | Chính xác |

|   |                                              |             |                                                                                   |           |
|---|----------------------------------------------|-------------|-----------------------------------------------------------------------------------|-----------|
| 4 | Flask được sử dụng để làm gì trong hệ thống? | Python.docx | Hệ thống trả lời đúng vai trò xây dựng giao diện web và xử lý yêu cầu người dùng. | Chính xác |
| 5 | Python là ngôn ngữ lập trình gì?             | Python.docx | Hệ thống đưa ra định nghĩa phù hợp với nội dung tài liệu.                         | Chính xác |
| 6 | Cơ sở dữ liệu là gì?                         | Python.docx | Hệ thống truy xuất và trả lời đúng khái niệm cơ sở dữ liệu.                       | Chính xác |
| 7 | Mô hình Client-Server hoạt động như thế nào? | Nhom3.pptx  | Hệ thống trả lời đầy đủ nguyên lý trao đổi dữ liệu giữa Client và Server.         | Chính xác |
| 8 | Google Gemini API có vai trò gì?             | Nhom3.pptx  | Hệ thống trả lời đúng vai trò sinh câu trả lời trong kiến trúc RAG.               | Chính xác |

### 3.6 Kết luận chương 3

Qua quá trình triển khai và thực nghiệm, đề tài đã xây dựng thành công hệ thống chatbot hỏi đáp tài liệu thông minh. Hệ thống đáp ứng tốt các mục tiêu đề ra ban đầu, bao gồm việc hỗ trợ đa dạng tệp tin (PDF, DOCX, PPTX), sở hữu giao diện web trực quan mô phỏng chatbot hiện đại. Quan trọng nhất, nhờ áp dụng triệt để kiến trúc RAG, sản phẩm có khả năng trả lời câu hỏi chính xác dựa trên tài liệu thực tế và khắc phục được nhược điểm bịa thông tin của các mô hình ngôn ngữ thông thường.



## KẾT LUẬN

### 1. Kết luận

Trong bối cảnh sự phát triển mạnh mẽ của trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên, việc xây dựng các hệ thống hỗ trợ tra cứu thông tin thông minh đang ngày càng trở nên cần thiết. Báo cáo đã trình bày quá trình nghiên cứu, thiết kế và xây dựng thành công hệ thống chatbot hỏi đáp tài liệu dựa trên kiến trúc Retrieval-Augmented Generation. Hệ thống được xây dựng với khả năng tiếp nhận và xử lý các tài liệu ở định dạng PDF và DOCX, thực hiện chia nhỏ văn bản, tạo vector embedding bằng mô hình SentenceTransformers và lưu trữ dữ liệu trong FAISS để phục vụ cho quá trình tìm kiếm ngữ nghĩa. Bên cạnh đó, việc tích hợp Google Gemini API giúp hệ thống có khả năng sinh câu trả lời tự nhiên, chính xác và bám sát nội dung tài liệu được cung cấp. Ngoài ra, hệ thống được triển khai trên nền tảng web sử dụng Flask kết hợp với HTML, CSS và JavaScript, giúp người dùng có thể dễ dàng tải tài liệu, đặt câu hỏi và nhận kết quả trực tiếp trên giao diện web. Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, tốc độ phản hồi tương đối nhanh và có khả năng cung cấp các câu trả lời phù hợp với nội dung của tài liệu. Tuy nhiên, hệ thống vẫn còn một số hạn chế như khả năng xử lý đồng thời nhiều người dùng còn hạn chế, chất lượng câu trả lời phụ thuộc vào nội dung tài liệu đầu vào và đôi khi vẫn có thể xuất hiện các phản hồi chưa thực sự chính xác. Trong tương lai, hệ thống có thể được phát triển theo các hướng như hỗ trợ thêm nhiều định dạng tài liệu khác, bổ sung chức năng trò chuyện bằng giọng nói, cải thiện khả năng ghi nhớ ngữ cảnh hội thoại và tối ưu hóa hiệu năng để phục vụ nhiều người dùng đồng thời. Những cải tiến này sẽ góp phần nâng cao chất lượng và mở rộng phạm vi ứng dụng của hệ thống chatbot trong thực tế.

### 2. Định hướng phát triển

Trong các nghiên cứu tiếp theo, hệ thống có thể được mở rộng và hoàn thiện theo nhiều hướng nhằm nâng cao hiệu quả và khả năng ứng dụng trong thực tiễn. Một số định hướng phát triển của hệ thống bao gồm:

- Hỗ trợ thêm nhiều định dạng tài liệu: Mở rộng khả năng xử lý đối với các định dạng như TXT, XLSX và các tài liệu dạng hình ảnh thông qua công nghệ nhận dạng ký tự quang học (OCR).
- Tối ưu hóa hiệu năng hệ thống: Cải thiện tốc độ xử lý và khả năng phục vụ đồng thời nhiều người dùng thông qua các giải pháp tối ưu mô hình và triển khai trên môi trường máy chủ chuyên dụng.
- Mở rộng khả năng triển khai thực tế: Phát triển hệ thống dưới dạng dịch vụ trực tuyến hoặc triển khai trên nền tảng điện toán đám mây để tăng khả năng truy cập và sử dụng.

- Nâng cao độ chính xác của hệ thống: Nghiên cứu và áp dụng các kỹ thuật RAG nâng cao, cải thiện quá trình truy xuất dữ liệu và giảm thiểu hiện tượng Hallucination trong các mô hình ngôn ngữ lớn. Những định hướng trên sẽ góp phần hoàn thiện hệ thống, nâng cao chất lượng câu trả lời và mở rộng khả năng ứng dụng của chatbot trong nhiều lĩnh vực khác nhau như giáo dục, doanh nghiệp và hỗ trợ tra cứu thông tin

## TÀI LIỆU THAM KHẢO

- [1] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd Edition Draft). Pearson.
- [2] Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th Edition). Pearson.
- [3] Lewis, P., Perez, E., Piktus, A., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
- [4] Reimers, N., & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [5] Johnson, J., Douze, M., & Jégou, H. (2019). "Billion-scale similarity search with GPUs". *IEEE Transactions on Big Data*, 7(3), 535–547.
- [6] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd Edition). O'Reilly Media.
- [7] Google AI. *Gemini API Documentation*. Truy cập tại: <https://ai.google.dev/gemini-api/docs>
- [8] Meta AI. *FAISS Documentation*. Truy cập tại: <https://faiss.ai/>
- [9] Sentence Transformers Documentation. Truy cập tại: <https://www.sbert.net/>
- [10] Flask Documentation. Truy cập tại: <https://flask.palletsprojects.com/>
- [11] Python Software Foundation. *Python Documentation*. Truy cập tại: <https://docs.python.org/>
- [12] LangChain Documentation. Truy cập tại: <https://python.langchain.com/docs/>
- [13] Google AI for Developers. *Google Generative AI Python SDK Documentation*. Truy cập tại: <https://ai.google.dev/>
- [14] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). "Attention Is All You Need". *Advances in Neural Information Processing Systems (NeurIPS)*.
- [15] Brown, T., Mann, B., Ryder, N., et al. (2020). "Language Models are Few-Shot Learners". *Advances in Neural Information Processing Systems (NeurIPS)*.